

Go — compiled path

Compiled from /home/dnikolic/Documents/Learning/Plan učenja/paths/Go . Canonical sources stay in place.

Phase — 01-go-fundamentals-and-cli-thinking

Directory: 01-go-fundamentals-and-cli-thinking/

Source: 01-go-fundamentals-and-cli-thinking/01-scope-outcomes-cadence-and-go-lab.md

Unit 1 — Program scope, mindset, and spine project (go-lab/)

What you should stop believing

You are not collecting syntax. Your target state is linguistic: **you stop reasoning “Go as dressed-up PHP classes”** and start reasoning **“Go as structs, behaviours on types, interfaces, pointers, composition, modules, explicit errors, and context”**.

Concrete end goal for Area 01 :

- **Struct, interface, pointer, error handling, package, module, context, receiver, composition, value vs pointer semantics**, and a basic sense of **who owns mutable memory**.
- **panic** is understood as exceptional, **not** the normal error path.

Suggested cadence (informational): roughly ten thematic blocks compatible with ~1–1.5 h/day is the original authoring pace. **Filenames order topics**, not calendars.

Continuous practice spine (Area 01)

For the entire area, evolve **one** CLI-shaped codebase called something like **go-lab/** :

- No web framework.
- No HTTP servers in Area 01 (defer transport to Area 05).

Types you carry forward: at least **User** , **Product** , **Order** , later a **Cart** , so every unit connects to earlier ones.

Capstone expectation (Unit 11): a separate small binary or subcommand tree **task-cli/** implementing **create**, **list**, **delete**, **complete** tasks. Use **structs**, **interfaces**, **explicit packages**, **go.mod** , **context** on any simulated slow work, disciplined **errors**.

How to document your learning

Prefer short **“why”** notes (constraints, alternatives rejected) beside code. Screenshots alone are worthless in review—they don’t expose thinking.

Final interview drill set for Area 01 (prepare spoken answers):

- interfaces (implicit satisfaction, narrowing),
- pointer vs receiver choice,
- **context** ,
- packaging,
- modules,
- **panic** vs idiomatic **error** flow,
- composition.

Unit 2 — Struct thinking: Go is not your PHP object model

Learning outcome

You model nouns primarily with `struct`, respecting **exported fields** versus **internal fields**, understanding **zero values**, and planning what “empty” means operationally—not magically “invalid until constructor runs” unless you deliberately enforce that elsewhere.

```
type User struct {
    ID    int
    Name  string
}
```

Core ideas

- A struct is ordinary data aggregated by type—not a deity class with obligatory factories (though you *may* add factories when invariants demand it).
- **Zero value:** `var u User` assigns `ID == 0` and `Name == ""`. That is predictable; your job is deciding whether those values are legally meaningful for your bounded context.
- Uppercase/lowercase identifiers still define **exported vs package-private** APIs at the declaration site.

Practice

Implement:

1. `User`
2. `Product`
3. `Order`

For each:

- Decide which fields tolerate zero values and where you will later forbid “half-built” aggregates (constructor function, validated builder, validation at command boundary).

Lab (“why” narration)

Explain in prose why `var u User` succeeds and is usable whereas in many PHP OO habits you subconsciously expected an “object lifecycle” ritual first. Tie that to predictable memory/layout and idiomatic constructors **only when semantics require them**.

Interview prompts

- struct layout intuition (ordering, alignment—high level okay),
 - zero value guarantees,
 - when zero is dangerous versus convenient,
 - how you’ll tag for JSON/SQL later without polluting domain rules today.
-

Unit 3 — Receivers: behaviour without inheritance

Learning outcome

You attach behaviour with `func (receiver) Method()` idioms while consciously choosing `(u User)` vs `(u *User)` receivers—because Go has **methods**, not subclasses.

Interpret:

```
func (u User) FullName() string
```

Understand **copy vs mutation**, **allocation vs aliasing**, and how receiver choice interacts with later **interface satisfaction**.

Value receiver vs pointer receiver (rules of thumb)

Use **pointer receivers** (`*T`) when the method meaningfully mutates `T`, when `T` is large and copying is wasteful each call, when you must preserve identity across callers, or when `T` carries mutexes/maps/slices backing shared state—consistency drives pointer choice far more often than micro-optimization.

Prefer **value receivers** only when semantics are immutable snapshot operations on small POD-like structs—or when you consciously want intentional copying for safety (rare discipline, not laziness).

Practice (cart)

Model a shopping-cart-like structure with behaviours:

- `Add(item ...)`
- `Remove(id ...)`

Select receivers deliberately; document mismatches (“if I mistakenly used value receiver here, duplication bugs appear like X”).

Lab

Produce a cheat-sheet paragraph **you would say in interview**:

- pointer receiver rationale,
- value receiver pitfalls on mutators,
- when interface satisfaction subtly forces pointer-ness.

Interview prompts

- receiver dispatch mechanism (non-OO mental model),
- copying slices/maps inside structs on value receivers,
- nil-pointer receiver edge cases (`var p *Cart` then calling method—know what breaks conceptually vs what oddly works depending on receiver type).

Unit 4 — Pointer semantics: ownership and sharing

Learning outcome

You read `&` (**address-of**) and `*` (**indirection**) mechanically *and* architecturally—you know whether a function observes **aliases** versus **temporary copies**.

This is traditionally where programmers coming from interpreted-first stacks misunderstand Go.

Sketch:

```
func UpdateUser(u *User) { ... }
```

Core ideas

- Passing `*User` shares identity of the backing struct with callers when they already possess a pointer; passing `User` duplicates bytes (note: structs containing pointers still share nested pointer targets—ownership stories compound).
- `UpdateUser` should make obvious **what it mutates**, what it forbids (`nil`), and what it returns (`error` discipline comes next formal unit but preview honesty here).

Practice

Implement `UpdateUser(u *User)` (name fields however you wish). Invent two call-site patterns:

- one where mutation is visible afterward,
- one where programmers *think* mutation happened but accidentally passed **values** copied into temp pointers or similar footgun.

Demonstrate corrected pattern.

Lab

Answer explicitly:

“Why might `UpdateUser(user)` apparently change nothing sometimes?” Narrate copying, pointer-to-temporary pitfalls, misuse of loops/range-variable addresses.

Interview prompts

- stack vs heap hand-waving discouraged—focus on observable semantics first,
- “when do I expose `*User` as API versus return new value?” pragmatics.

Source: `01-go-fundamentals-and-cli-thinking/05-interfaces-implicit-contracts.md`

Unit 5 — Interfaces: behavioural contracts without inheritance drama

Learning outcome

You treat interfaces as descriptions of capabilities—**often tiny**—rather than hierarchies proclaiming ontology.

Starter shape:

```
type Logger interface {  
    Log(message string)  
}
```

Implement independently:

- `ConsoleLogger`
- `FileLogger`

Inject them wherever domain actions need auditing—avoid global singletons early so later tests sting less.

Why “small interfaces” win

Compose systems from replaceable collaborators. Oversized interfaces become mock-the-universe nightmares and hide unused contract corners.

Classic interview phrasebook item: **interface segregation** interpreted operationally—“don’t merge unrelated behaviours just because naming is convenient Tuesday morning.”

Implicit satisfaction means your concrete types declare **nothing** tying them to interfaces except actually implementing listed methods—that decouples packages.

Practice

Route logging decisions through `Logger` so CLI actions can silence output, tee to `stderr`+file later, swap with `io.Writer` façade when you deem proper.

Lab

Write rationale for narrowing `Logger`: what did you consciously **not** add (levels? structured fields?), and deferral rationale.

Interview prompts

- implicit vs explicit implementations (Dart/Java contrast),
- when huge interface from `stdlib` emerges (`io.Reader`) vs when you forbid local mega-interfaces.

Source: `01-go-fundamentals-and-cli-thinking/06-errors-and-responsibility.md`

Unit 6 — Error handling thinking: responsibilities, wrapping, panic discipline

Learning outcome

Treat Go errors as ordinary **first-class returned values**:

```
value, err := Do()
```

You practise:

- `errors.New`
- `fmt.Errorf` with verbs including `%w` when layering failure causes expect introspection downstream.

Understand **defer** / **recovery** folklore only after recognising `panic` is rarely your happy-path instrument.

Go **does not** elevate exceptions controlling normal outcomes; pretending it does produces unreliable services quietly.

Core ideas

- Every failure path chooses **classification**: caller-retry-able? programmer bug? sanitation for user-visible message?
- **Wrap** sparingly yet meaningfully—“open config: ...” beats anonymous `return err`.
- `panic` signals **stop the world-ish** invariant violation or impossible states after programming mistakes—not missing files in CLI unless you consciously scope recovery at `main`.

CLI practice sketch

Exercise file operations (read absent file, unreadable permission). Thread errors upward enriching context at boundaries.

Contrast:

- returning wrapped errors,
- logging then swallowing (**forbidden habitual pattern** articulation),

Lab

Document three scenarios:

1. **network-ish** ephemeral errors (simulate),
2. **validation** misuse,
3. **programmer** misuse.

Argue succinctly panic appropriateness (likely none—but justify edge cases responsibly).

Interview prompts

Topic	Probe
panic vs error operational culture	articulate
sentinel errors introduction (errors.Is)	bridging future modules
when not to wrap (avoid chaining noise pollution)	

Source: 01-go-fundamentals-and-cli-thinking/07-package-layout-cmd-internal-and-pkg.md

Unit 7 — Package layout (cmd , internal , guarded pkg)

Learning outcome

Internalise `cmd/` binaries, cohesive domain packages (`user/` , `order/` , `config/` , etc.—names illustrative), sparing third-party-esque `pkg/` , and restrictive `internal/` enforcement—so import graphs stay honest.

Ownership question always: “**Who may depend on whom, and why?**”

Canonical mental map

```
repo/
├ cmd/...           ← thin mains wiring dependencies
├ internal/...      ← not importable by external modules
└ <domain pkgs> ← reusable-but-not-published libraries if internal not appropriate
```

Practice refactoring go-lab/

Migrate from monolith file sprawl progressively:

- Separate domain types & behaviours cleanly.
- Isolate configuration parsing so flags/env handling doesn’t haunt business logic accidentally.

Forbidden dumping ground temptation: miscellaneous `helpers` package—articulate refactor plan when you drift there.

Lab

Draw import DAG (ASCII okay). Locate **potential cycles**. If hypothetical cycle emerges, narrate dissolution strategy splitting interfaces / moving interfaces consumer-side thoughtfully.

Interview prompts

- difference between `internal` enforcement vs social convention folders,
 - when breaking packages prematurely hurts velocity vs premature monolith viscosity,
 - cohesion vs duplication boundaries.
-

Unit 8 — Modules: reproducible graphs and dependency honesty (`go.mod`)

Learning outcome

You operate `go mod init` + `go mod tidy` fluently—not mystically—including understanding **semver intent**, pinning vs floating trade-offs for learning reproducibility.

Add exactly **one** legitimate third-party library solving a mundane problem (**CLI flags**, **YAML/TOML config**, hashing/UUID—you choose) documenting:

- rationale,
- behavioural surface you rely on.

Concepts to internalise

- **Module path uniqueness** anchors import strings.
- **Sum file integrity** reasoning (supply chain conscience).
- **Pseudo-versions / retracted versions**: awareness—they impact upgrade paths.

Avoid dependency shopping sprees—they expand attack surface mentally and operationally even if scanners quiet today.

Lab written answer

Produce short risk note:

- hypothetical upgrade breakage scenario,
- how you'd bisect regressions responsibly (`go.mod` pinning strategies at learning scale).

Interview prompts

- minimal dependency philosophy pragmatic cases,
- `replace` directives usage cautions (`go.mod` escapes),
- private module proxy awareness (conceptual readiness).

Unit 9 — Context: timeouts, cancellations, disciplined propagation—not globals

Learning outcome

Treat `ctx context.Context` as **deadlines + cancellations + bounded values (sparingly)** threading cooperatively—not a mystical bag chucking incidental configuration everywhere.

Mandatory internalisation sketches:

```
ctx, cancel := context.WithCancel(parent)
defer cancel()
```

```
ctx, cancel := context.WithTimeout(parent, 2*time.Second)
defer cancel()
```

Misuse anti-patterns to articulate

Stuffing unrelated config arbitrarily into `context.Value` without typed accessor discipline.

Propagating `context.TODO()` permanently—document only transitional scenarios.

Ignoring `ctx.Done` in loops performing simulated network or IO.

CLI practice blueprint

Implement a **download-like** faker (sleep or read large file concurrently) enforcing **overall budget** shorter than faker duration—observe clean abort vs leaky goroutine tendencies (note leak—fix responsibly even if minimally).

Highlight **why context is not global singleton state**.

Interview prompts

- parent → child propagation mental model diagrams,
- when context cancellation should preempt vs escalate metric logging,
- relationship with upcoming HTTP inbound deadlines (Area 05), DB calls (Area 08), gRPC (Area 06).

Source: `01-go-fundamentals-and-cli-thinking/10-composition-for-capabilities.md`

Unit 10 — Composition: inheritance intuition reframed deliberately

Learning outcome

Rebuild **capabilities** assembling structs embedding collaborators / explicit fields instead of phantom inheritance hierarchies.

Construct:

```
OrderService + Logger collaborator + injected Config abstraction
```

Clarify how PHP mental models tempt **deep derivation** stacks that obscure dependency directions.

Embedding nuance checklist

Exported promoted methods readability vs surprise—discipline naming so promoted API surface remains intentional—not accidental shotgun.

Practice

Wire `OrderService` constructor accepting collaborators via interfaces/concrete collaborators where justified.

Show **alternate composition** rewriting a hypothetical naive inheritance doodle from PHP-think into idiomatic decomposition.

Lab

Articulate succinctly inheritance advantages you surrendered consciously **and what Go compositional ergonomics reimbursed you** regarding testability/import clarity.

Interview prompts

- embedding vs plain field trade-offs,
- when shallow embedding becomes API leak,

- mocking/service substitution via interfaces interplay preview (Area 09),
- synergy with structs/receivers (Units 2–4).

Source: 01-go-fundamentals-and-cli-thinking/11-capstone-task-cli-integration.md

Unit 11 — Capstone: task-cli integration proving Area 01 themes

Build target

Produce task-cli/ (standalone main acceptable) supporting:

Command semantics	Behaviour
create	Stores new task identifiable later.
list	Shows tasks with status fields you designed.
delete	Removes logically or tombstones consciously—explain choice briefly.
complete	Marks progression with consistent invariants (completed_at ?).

Underlying storage can be naive JSON/files while learning—**explain trade-offs** anticipating Area 08 seriousness later.

Apply deliberately:

Theme	Artefact reviewers expect
struct	expressive domain types
interface	seams (repos, clocks, printers) narrow where helpful
pointer/value choices	articulated in short design note
context	at least one bounded operation cancelling/timeout cleanly
package split	cohesion story over single-file sprawl unless justified temporary
go.mod	tidy dependency rationale
error handling	no silent ignores; layering clarity

Deliverable narratives

Produce documentation emphasising WHY not giant code dumps exclusively:

1. Ownership chart (bullet list).
2. Panic-vs-error rationale statements.
3. Pointer/receiver decisions summarised succinctly referencing earlier units.

Oral rehearsal checklist (**must be smooth unscripted-ish**):

- implicit interfaces,
- pointer semantics anecdotes,
- receiver reasoning,
- context placements,
- module hygiene,
- panic vs error ,
- composition story.

Phase — 02-go-tooling-engineering-workflow

Directory: 02-go-tooling-engineering-workflow/

Source: 02-go-tooling-engineering-workflow/01-formatting-linting-and-task-runners.md

Unit 1 — Toolchain rhythm: formatting, lint, vet, repeatable tasks

Suggested informational cadence: two thematic blocks aligning with roughly two abbreviated “segments” originally described as tooling weeks—not strict calendar commitments.

Learning outcomes

Operational habits settle so you reflexively normalize code before review—not after argument:

Tool	Role
go fmt	canonical indentation / simplicity standard
goimports (or editor integration aligning to it)	import grouping + pruning unused
golangci-lint aggregator	umbrella running many analysers—you configure gradually consciously
staticcheck	higher-signal correctness + deprecation insight
go vet	suspicious constructs early

Reproducibility via Makefile or Taskfile exposing targets like:

```
fmt tidy vet lint test build
```

Practice spiral

Extend your go-lab / task-cli repository:

```
CLI change → golangci-lint → fmt/tidy gate → deterministic build artefact naming
```

Interview prompts

Why CI duplication of editor format-on-save beats “trust humans”. When overly aggressive linters rot velocity—balancing signal vs noise articulated.

Acceptance criteria

Demonstrate scripted sequence (conceptually documented if automation environment constrained elsewhere) reproducible identical commands invoked locally + CI analogue sketch.

Unit 2 — Delve debugging, build tags, `embed`, cross-build, cautious CGO edges

Learning outcomes

- Launch **delve** sessions—breakpoints, stepping, goroutine/stack inspection—not merely `fmt.Printf` archaeology when complexity spikes.
- Use **build tags** toggling backends / integration-heavy files without `#ifdef` chaos—predictable `-tags` combos documented.
- Package static assets thoughtfully using `//go:embed` (scope discipline: don't inflate binaries absurdly unknowingly).
- Produce **Linux + Windows** (or analogous pair) artefacts through `G00S` / `GOARCH` awareness.
- Respect **pure-Go** portability vs `cgo` introducing harder cross-compile + deployment coupling—articulate pragmatic trade-offs.

Practice sequence

Conceptual rehearsal even if constrained:

```
linux artefact compile → alternate OS compile → seeded bug breakpoint fix narrative
```

Interview prompts

Contrast debugging concurrent code vs sequential—breakpoint discipline hazards.

When `cgo` becomes unavoidable (hypothetical cryptography / libs) mitigation strategies (builder images, pinning).

Acceptance criteria

Produce short written debugging story referencing **goroutine vantage** inspecting stuck worker scenario hypothetical.

Source: 02-go-tooling-engineering-workflow/03-linux-and-network-process-triage-for-backend-debugging.md

Unit 3 — Linux + network triage beside Delve: processes, sockets, and syscalls

Learning outcomes

- Map a **listening port** (`LISTEN`) to a **PID** and binary without guessing—which process actually owns `:8080`.
- Inspect **TCP connection state** summaries when backends look “wedged” (many `CLOSE_WAIT`, saturation, asymmetric counts).
- Use `strace` / `perf` at a restrained, read-only scope to distinguish “blocked on I/O syscall” vs “burning CPU in userspace”—without treating attach-to-prod lightly.
- Read `/proc/<pid>/` essentials (`fd`, `limits`, brief `stack` awareness) when `delve` is unavailable (stripped binaries, hostile environments).

Concepts to internalise

- **Observability vs debugging:** syscall traces prove behaviour; structured logs/traces explain intent—use each where it earns its blast radius.
- **Production etiquette:** syscall attachment can stall high-throughput processes—prefer replicas, lowered traffic reproduction, or canary namespaces when possible.

Lab (local or disposable VM)

Pick a toy HTTP listener (your earlier CLI or minimal `chi/net/http` service). Without opening the codebase first:

```
identify PID + binary path
enumerate listening sockets correlating fd → socket
simulate slow client behaviour and observe backlog / state changes narratively (even if tooling output only described in prose)
```

Interview prompts

- When would you **strace** a Go binary vs **go tool pprof** vs **tracing** vs **TCP dumps** (`tcpdump` only where policy allows)?
- How does **ephemeral-port exhaustion** mimic app bugs?

Acceptance criteria

Short written note contrasting **three** hypothetical symptoms (high CPU, stalled requests, intermittent timeouts) → **which** Linux/tool signal you'd check first → **why** Delve alone might lie.

Phase — 03-go-concurrency-and-queue-workers

Directory: 03-go-concurrency-and-queue-workers/

Source: 03-go-concurrency-and-queue-workers/01-scope-capstone-queue-worker.md

Unit 1 — Concurrency mindset and spine project (queue-worker/)

Suggested informational cadence: ten thematic segments originally paired roughly with daily ~1–1.5 h pacing—**topic order preserved**, not timetable enforcement.

Learning outcome trajectory

Discard vague “threads scary” folklore for **purposeful parallelism mental model**:

Primitive	Operational meaning
goroutine	lightweight logically concurrent function execution scheduled cooperatively—not free infinite scaling token
channel	communicate to share memory (idiom reversal from classical locks-first habits)
mutex	explicit shared-structure protection discipline
select	readiness multiplexing—not busy polling camouflage
context	cooperative cancellation propagation—again—not magic globals

Understand **ownership of shutdown**: leaking goroutines is a correctness defect—even if demos “appear fine” briefly.

Final Area 03 capstone (**Unit 11**): integrate **producer**, **bounded buffer/channel**, **worker pool**, **select** coordination (timeout/cancel readiness), graceful **shutdown** narrative, intentional **fault injection** arcs (later repaired).

Interview consolidation later must cover:

goroutine distinctions vs OS threads coarse mental model • channel buffering trade-offs • **select** patterns • **context** cancellation propagation • **sync.Mutex** under contention • **go run -race** signalling data races • worker pool structuring.

Spine repository

Maintain **queue-worker/** codebase (no frameworks, no mandatory HTTP)—pure concurrency educational laboratory across Units 2–11.

Acceptance ethos: reproducible behavioural commentary plus code—not mystical “trust me parallelism” assertions.

Source: 03-go-concurrency-and-queue-workers/02-goroutines-and-nonblocking-launch.md

Unit 2 — Goroutine thinking: cooperative scheduler intuition

Learning outcome

Internalise **go func()** separates *launch scheduling* vs *completion witnessing* demands explicit synchronization primitives (**WaitGroup**, channel closure patterns, supervised **errgroup** patterns later—you choose minimal honest mechanism now).

Interpret Go mental model contrasts:

Bad oversimplification: “thread == goroutine simplistic manual lifecycle” → misleads newcomers.

Closer truth: goroutines multiplex atop runtime scheduler cooperating with cooperative blocking semantics at channel / syscall boundaries—study surface remains **logical concurrency clarity first**, internals second.

Practice

Simulate “email/job dispatch”: fan out ~5 concurrently operating workers printing traceable IDs respecting ordering non-guarantees—observe nondeterministic interleaving deliberately.

Explain conversationally:

Why `go Process()` does not block callers until you add coordination.

Lab question (answer aloud)

Enumerate typical beginner bugs:

- orphaned goroutines,
- main exiting early,
- unbounded spawning spirals unrealistic production analogues.

Interview prompts

Cheap creation $\neq$ cheap system load under pathological spawning.

Preemption evolution awareness (fine-grained preemption matured historically—don’t memorize timeline obsessively unless interviewing kernel teams).

Source: `03-go-concurrency-and-queue-workers/03-channels-share-by-communicating.md`

Unit 3 — Channel idioms: “Don’t communicate by sharing memory blindly” orientation

Learning outcome

Adopt proverb directionally—even if nuanced later—for teaching clarity:

Prefer **ownership passing** idioms leveraging `chan` over ad-hoc global maps protected casually.

Starter pattern:

```
jobs := make(chan int)
```

Produce **producer** fabricating simulated tasks; consumers drain executing slower operations (sleep mocking latency).

Understand **blocking semantics** zero-buffer channels: synchronization rendezvous pairwise readiness.

Practice tasks

Implement minimal pipeline:

```
task creator goroutine → channel → executor goroutine
```

Articulate buffering absence consequences on throughput vs deterministic handoff choreography.

Lab

Contrast readability / failure surface vs unstructured shared structs mutated under `time.Sleep`-based gambles—not production patterns.

Interview prompts

Directed channel types (**send-only**, **receive-only**) in function signatures signalling intent cleanly.

Source: `03-go-concurrency-and-queue-workers/04-buffered-channels-capacity-tradeoffs.md`

Unit 4 — Buffered channels: capacity as temporal slack—not free lunch

Pattern:

```
make(chan Job, 10)
```

Learning outcomes

Understand **producer** → **buffer** → **receiver** pacing decouples bursty arrivals partially while introducing:

- fullness blocking producers,
- emptiness blocking consumers,
- potential **lost pressure signals** forgetting shutdown coordination.

Capacity choice is behavioural architecture—not magic constant `10`.

Practice

Maintain bounded backlog ~10 queued fake jobs witnessing producers temporarily quicker than lazy consumer—observe growing occupancy—then choke consumer entirely narrating starvation vs backlog growth consequences qualitatively.

Lab comparative writing

Enumerate latency / throughput intangible trade-offs **buffer vs unbuffer**.

Interview prompts

Danger: mistaken belief buffered channels eliminate need for disciplined cancellation.

Deadlocks when both sides blocked incorrectly—diagram mental fix.

Source: `03-go-concurrency-and-queue-workers/05-select-multicase-coordination.md`

Unit 5 — `select`: multiplex channel readiness cleanly

Learning outcome

Use `select` combining:

- progressing work readiness,
- **timeouts** driven by `time.After` idioms cautiously—or better integrate context cancellation next units holistically revisiting timeouts,
- **shutdown** signalling channel closed intentionally.

Understand pseudo-random fairness among simultaneous ready multi-case completions—avoid brittle ordering assumptions exploiting races.

Practice

Unified worker loop juggling:

```
workCh
shutdownCh / done context
possibly watchdog timers
```

Lab narrative obligation

Articulate consciously why naive **sleep-loop polling** corrodes readability + masks missed wakeups subtly.

Interview prompts

Compose vs sequential channel reads patterns.

Dangerous **default** selects busy-spin regressions unintentionally resurrecting CPU waste.

Source: `03-go-concurrency-and-queue-workers/06-context-cancellation-hierarchies.md`

Unit 6 — Context cancellation hierarchies propagate termination intent

Patterns:

```
ctx, cancel := context.WithCancel(parent)
defer cancel()
```

Interpret **parent** ⇒ **child** ⇒ **grandchildren** subgraph cancellation semantics when parent fire cancellation—downstream observes `Done` closures.

Contrasts mistaken mental model stacking detached `context.Background()` trees ignoring propagated coordination—enterprise Go begins structuring cross-service boundaries conscientiously—even if exercise limited locally first.

Practice extensions

Enhance `queue-worker/` scaffolding: cancelling long faked jobs cooperatively—not merely killing goroutines brutishly.

Mandatory reflection writing

Demonstrate cancelling propagates cleanly without lingering blocked senders mistakenly—coordinate channel closure thoughtfully.

Interview prompts

Always defer cancel discipline reasoning.

Context misuse stuffing unrelated config overshadowing readability—disciplined restraint.

Source: 03-go-concurrency-and-queue-workers/07-timeouts-payment-fakes.md

Unit 7 — Timeout discipline: simulated slow dependencies respect budgets

Learning outcome

Bind external-ish latency envelopes using `context.WithTimeout` consciously contrasting unbounded sleeps spelling operational doom.

Practice scenario:

- faker “payment gateway” delaying **5 s intentionally** while consumer policy dictates ≤ 2 s overall budget.

Document resulting `context.DeadlineExceeded` style handling path plus logging categorisation distinctions (transient vs caller misuse vs dependency SLA breach).

Cross-link anticipating distributed HTTP timeouts later (**Area 12**) philosophically—even if code stays local here.

Lab

Enumerate timeout placement strategies distinguishing:

- coarse per overall job,
- nested inner budgets tightened iteratively responsibly.

Explain misplacement causing duplicate cancellation confusion.

Interview prompts

Propagating shortening child contexts vs independent sibling contexts—diagram narrative verbally.

Source: 03-go-concurrency-and-queue-workers/08-mutexes-and-shared-mutable-state.md

Unit 8 — Mutex & shared mutation: pessimistic guarding when purity cracks

Bad habit anti-pattern caricature:

“Just share maps across goroutines because map writes feel easy.” → **crash / data race chaos**.

Responsible pattern:

Synchronize with `sync.Mutex` guarding invariants—not sprinkling blindly after-the-fact when `-race` yells louder than design.

Practice

Increment shared counter pressured by ~100 concurrent goroutines.

1. Demonstrate flaky wrong results without instrumentation (may appear “fine” sporadically—never trust nondeterministic green runs).
2. Re-run enforcing `-race` highlighting data races vividly.
3. Serialize increments via mutex concluding determinism reproducibly—also measure coarse latency inflation qualitatively (not microseconds wars yet).

Lab written reflection

Enumerate **** Mutex vs channels**** decision heuristics: linear ownership pipelines vs shared counter-style hotspot needing lock.

Discuss **ordering deadlocks**: acquiring multiple locks inconsistently lethal pattern—sketch hypothetical cross-lock scenario resolved by consistent acquisition partial ordering conventions.

Interview prompts

Contrast **optimistic concurrency** glimpses forward (defer Area 08 DB depth) layering mental hooks.

Source: 03-go-concurrency-and-queue-workers/09-worker-pool-enterprise-pattern.md

Unit 9 — Worker pool structuring: amortise goroutines deliberately

Corporate Go systems rarely spawn unbounded ephemeral goroutines for each remote event—economics degrade under fan-in storms.

Prefer **bounded worker goroutine set** draining shared job intake channel responsibly.

Practice assignment:

Design **email-ish sender** analogue:

```
1000 jobs total
≤ 5 simultaneous active workers draining queue
shutdown waits in-flight politely honoring context cancellation tightening
```

Learning outcomes articulated

Enumerate **graceful draining** choreography:

- cease accepting inbound,
- cooperative completion,
- join / wait layering.

Articulate **backpressure**: sluggish workers imply queue growth—eventually producers must choke, throttle, or shed load—explicit policy decision not accidental memory ballooning unnoticed.

Interview prompts

Sizing worker pools via metrics-informed loops vs arbitrary constants—risk articulation.

When pool too small induces SLA misses vs oversized spinning idle CPU waste.

Source: 03-go-concurrency-and-queue-workers/10-race-detector-drills.md

Unit 10 — Race detector: instrument, provoke, refactor—don't shrug nondeterminism

Operational command habits

Embed `go run -race` / `go test -race` into learning loops—even if slowdown painful—prefer catching unsynchronised accesses early morally cheaper than flaky prod heisenbugs exhausting teams.

Go race detector observes **happens-before** instrumentation approximations—still invaluable teaching discipline.

Practice cycle

Purposefully synthesize deterministic race regressions—you know root cause consciously—observe detector reports reading until clarity appears—repair via:

- narrower ownership pipelines,
- channel handoff rewriting,
- explicit mutex guarding chosen deliberately.

Forbidden outcome: patching sleep durations hoping races vanish probabilistically—that’s folklore debugging punishment later.

Lab deliverable checklist

Brief write-up interpreting race report lines mapping to offending source lines—even if abbreviated paraphrasing.

Interview prompts

Instrumentation overhead magnitude expectation order-of-mouse-not-science-ballpark honesty.
Contrast property-based concurrency tests vs `-race` complementarity philosophies.

Source: `03-go-concurrency-and-queue-workers/11-capstone-queue-worker-integration.md`

Unit 11 — Capstone integration: disciplined `queue-worker/` journey

Compose features holistically—not isolated toys:

```
producer emits → bounded buffer/channel → worker pool draining
context cancellation/timeouts cooperating
shutdown discipline narrated plainly
race detector cleanliness targets (no knowingly ignored diagnostics)
```

Fault injection arcs (ethical “break then heal” rehearsal)

Produce **alternate scratch branches or tagged commits capturing intentional pathologies**:

pathology	pedagogical payoff
deadlock	lock/channel ordering cognition
goroutine leak	forgotten join / ignoring cancellation
data race	detector literacy + refactor

Heal each cleanly documenting **cause → structural fix rationale**—“because `sleep(10ms)` waved problem away” disqualifies academically.

Interview consolidation emphasises:

```
goroutines • channels (+buffering contrasts) • mutex • select contexts
timeouts • cancellations • -race • worker pooling • graceful shutdown choreography
```

Deliverables

runnable reference path README chunk emphasising WHY boundaries chosen—not merely command invocation screenshots.

Phase — 04-go-concurrency-advanced-labs

Directory: 04-go-concurrency-advanced-labs/

Source: 04-go-concurrency-advanced-labs/01-races-syncmap-and-shared-maps.md

Unit 1 — Shared-map turbulence: practise mutex, `RWMutex`, and `sync.Map` trade-offs

Bridging informational cadence: conceptual block sequence originally mapped to roughly four deepening “weeks”; treat numbering as deepening order.

Practice scaffolding

Stress **shared map** concurrent writes until runtime panics or races manifest—observe failures differ under `-race` harness vs naive runs.

Remediation ladder:

Stage	Tooling	Commentary
1	<code>sync.Mutex</code> around every mutating structural access	straightforward baseline
2	<code>sync.RWMutex</code> distinguishing dominant read-heavy profile	amortise contention when truly read skewed—not cargo-cult prematurely
3	<code>sync.Map</code> exploratory fits specialized dynamic key churn / cache-ish patterns—not drop-in synonym for careless global map substitution	

Articulate drawbacks: coarse lock serialisation vs fine-grained design complexity trade spectrum.

Cross-reference eventually **PostgreSQL concurrency** grounding truth—prefer DB invariants authoritative when domain demands durability—not heroic in-memory map warfare.

Deliverable expectation: reproducible storyline + metrics-free qualitative latency commentary permissible now.

Source: 04-go-concurrency-advanced-labs/02-worker-pool-fan-out-fan-in.md

Unit 2 — Worker pipelines: fan-out / fan-in + backpressure awareness

Elevate pooling toward pipeline topology:

source → replicated workers (fan-out) → aggregation join (fan-in)

Learning outcomes

Maintain **explicit backpressure**:

- Bounded intermediate channels preventing unbounded intermediary memory explosion when stages skew rates.

Enumerate completion signalling strategies preventing aggregator goroutine stalls silently.

Practice sketch—even if artificially small dataset—articulating hypothetical metrics hooking placeholders for future Prometheus instrumentation bridging ops mindset elsewhere in vault optionally.

Interview lenses map to enterprise throughput debugging stories verbally even if hypothetical.

Acceptance criteria checklist

Narrative diagram (ASCII permissible) illustrating data motion + choke points—not single mystery function monolith intangible.

Source: 04-go-concurrency-advanced-labs/03-context-cancel-timeout-deadline-mastery.md

Unit 3 — Context mastery layering: Cancel / Timeout / Deadline interplay

Operational trio internalisation:

```
context.WithCancel
context.WithTimeout
context.WithDeadline
```

Fabricate illustrative chain approximating mythical:

```
incoming HTTP-ish handler context (simulate)
  → simulated DB repository operation inheriting shrunk budget
  → nested gRPC-esque fake client call inheriting tightened cap again
```

No full frameworks obligatory—minimal structural fakes suffice if they honour Done propagation truthfully.

Lab essay topics

Enumerate misconfigurations collapsing budgets unintentionally—notably spawning fresh Background() severing ancestry accidentally.

Articulate sentinel error readability mapping context.DeadlineExceeded handling vs generic error stringContains hacks historically frowned upon.

Interview expectation: verbally diagram tree cancellation waves like ripples succinctly—not elaborate animation—just coherence.

Source: 04-go-concurrency-advanced-labs/04-leaks-deadlocks-unblocking-drills.md

Unit 4 — Leaks, deadlocks, blocked-channel forensic drills

Controlled destructive exercises—then heal:

defect class	trigger sketch	remediation principle
goroutine leak	spawned worker ignoring parent ctx.Done() perpetually loops	unify lifecycle join
channel block mismatches	forgotten receiver / closed prematurely mis-synchronisation	unify state machine choreography
deadlock	inconsistent lock/channel acquisition ordering	deterministic acquisition policy / refactor pipeline

Produce micro-postmortem bullet lists per class emphasising preventative heuristics teachable verbally under interview hypothetical debugging prompts.

Interview storytelling goal: interviewer describes symptom logs—you outline diagnostic narrowing steps without insisting single silver-bullet tool blindly.

Phase — 05-http-apis-service-ownership-and-chi

Directory: 05-http-apis-service-ownership-and-chi/

Source: 05-http-apis-service-ownership-and-chi/01-scope-go-api-capstone.md

Unit 1 — Scope: go-api/ service spine (HTTP as real backend craft)

Informative cadence note: original author envisioned ~10 deepening segments at ~1-1.5 h blocks—**topic sequencing only**.

Learning outcome shift

Stop equating “backend” with “I exposed JSON”. Start treating systems as **living request lifecycles**:

```
socket accept → mux/router → middleware stack → thin handler adapters
→ application behaviours → collaborators (repos/clients/logging)
→ explicit failure mapping → structured observability breadcrumbs
→ deadlines/cancellation coherence
```

Stack anchors (purposeful restraint)

Implement go-api/ primarily using:

Layer	Guidance
Transport	net/http foundational clarity supplemented by chi routing ergonomics—not opaque macro-framework
Serialisation	encoding/json explicit control
Validation	go-playground/validator (or comparable explicit strategy) guarding hostile inputs

Defer ORM fantasies deliberately—integrate Postgres explicitly later (Area 08) aligning sqlx philosophy.
Interview mindset emphasises **ownership**: each layer declares **what failures mean** externally vs internally—not accidental leaking stack traces verbatim.

Source: 05-http-apis-service-ownership-and-chi/02-http-handler-lifecycle.md

Unit 2 — HTTP handler lifecycle fundamentals (net/http mental clarity)

Interpret HTTP not as “ /users ⇒ JSON magically” caricature—but orchestrated interplay:

```
Request semantics (method/path/headers/body/stream)
⇒ Middleware transformations (ordering matters)
⇒ Handler adapter glue
⇒ Response writer semantics + status/header/body choreography
⇒ Deferred resource closure responsibilities
```

Practice warmup

Initially optional bare ServeMux / HandleFunc if clarifies layering—migrate promptly toward disciplined router structuring next unit bridging chi.
Expose:

```
GET /health (200 minimal JSON heartbeat)
GET /users (temporary slice static / or stub eventual repository seam)
```

Lab reflection

Enumerate **panic recovery** layering discussion preview—defer full patterns until middleware unit but conceptual placement begins now.

Articulate pitfalls mutating **http.Request** context replacement misuse historically tripping juniors (`r.WithContext` disciplined usage later intensifies).

Interview checklist item: summarise **lifecycle** verbally without textbook quote regurgitation.

Source: `05-http-apis-service-ownership-and-chi/03-chi-router-shaping-apis.md`

Unit 3 — Router shaping (`chi`): resources, verbs, `{id}` parameters

Practice route map scaffold

Establish coherent resource tree aligning REST-ish pragmatics (don't feud pedantic REST theology yet—focus cleanliness):

```
GET /health
GET /users
POST /users
/users/{userID}
GET /orders
...
```

Choose parameter naming bridging `chi` patterns (`chi.URLParam` etc.) responsibly—document uniformity conventions early preventing drift.

Routing layer remains **thin**: parameter extraction + delegating—not SQL embed hideouts.

Interview dimension: Discuss **ownership** delineation—mux knows path shapes; handlers adapt to typed service calls—not deeply nested closures anonymous obscuring coverage.

Source: `05-http-apis-service-ownership-and-chi/04-dtos-handler-service-boundaries.md`

Unit 4 — Request/response boundaries: handlers stop being database scripts

Anti-pattern diagnosis

Flattened handler descending raw SQL monstrosities interleaved JSON marshalling + domain rules—“works day one, rots iteration two.”

Prefer explicit adaptation layer:

```
HTTP DTO/request shape → validated domain/command structure → persisted model distinctions if justified
```

Practice trajectory

Implement `POST /users /CreateUser` style flow:

- Incoming JSON → **request DTO**.

- Transformation / validation segregation upcoming unit intensifies interplay.
- Outgoing response shaping intentionally omitting leaky internal fields exposing DB surprises.

Interview lens contrasts **transport models** versus **persisted aggregates** thoughtfully—not dogmatic cloning always—just conscious mapping.

Source: 05-http-apis-service-ownership-and-chi/05-validation-with-playground-validator.md

Unit 5 — Validation discipline: mistrust payloads structurally (`validator` toolkit)

Practice endpoint hardening **POST /users** (extend analogously orders/products iterating forward).

Typical validations:

required • string length sane bounds • basic email lexical pattern • numeric ranges

Articulate layering:

Validation tier	Responsibility
structural / schema	tags / explicit checks
business invariants deeper	possibly service enforcing cross-field rules

Lab adversity testing

Fabricate purposely malicious JSON bodies provoking:

400 coherent problem detail (shape stable)
versus leaking internal sentinel strings carelessly deprecating reviewer trust.

Interview emphasis: distinguishing **caller misuse** (4xx classification) versus **unexpected internal defect** disguised clumsily.

Source: 05-http-apis-service-ownership-and-chi/06-middleware-cross-cutting-stripes.md

Unit 6 — Middleware cross-cutting fibres: tracing hooks, audit, auth placeholders

Learning outcome catalogue

Treat middleware onion ordering intentionally:

Suggested outer→inner illustrative ordering (adapt consciously):

panic/recovery sentinel (thin)
logging / correlation id enrichment
authentication / authorization scaffolding (possibly stub verifying header presence minimally)
metrics future hook comment anchors
routing entry

Practice capabilities:

request id attribution (propagate contextual logging downstream)
rudimentary structured logging scaffolding bridging next unit amplification
opaque future auth verifying static bearer token minimally if time permits—not security theatre prete
nding completeness—label clearly educational stubbing

Lab essay

Articulate distinctions **middleware vs handler** responsibilities preventing domain logic creep upward accidentally.

Enumerate danger compressing heavyweight DB transactions inside middleware—justify rarely exceptional cases if any—even hypothetical negatives primarily.

Source: 05-http-apis-service-ownership-and-chi/07-structured-logging-production-shape.md

Unit 7 — Structured logging readiness (`slog` or `zap`)

Transition motivation

Eliminate habitual `fmt.Println` tracing masquerading observability—it collapses grep-ability correlating outages. Prefer keyed fields aligning:

```
logger.Info("creating order",
  "request_id", requestIDFromCtx,
  "user_id", userID)
```

Practice linking **middleware-injected identifiers** bridging each log emission minimum baseline contract.

Fabricate deliberate **internal 500** scenario verifying discoverability slicing logs later conceptually—even absent full ELK stack integration now.

Interview angles:

- cardinality cautions injecting high fan-out dynamic labels exploding metric/log storage costs unintentionally glimpsed verbally,
 - PII scrubbing instincts even in learning exercises ethically.
-

Source: 05-http-apis-service-ownership-and-chi/08-error-wrapping-and-http- semantics.md

Unit 8 — Error wrapping + purposeful HTTP semantics (no silent bubbling)

Practice pattern emphasis:

```
fmt.Errorf("create order: %w", err)
```

Then boundary translation mapping internal errors selectively:

```
domain/repository distinctions → sanitized external API errors
versus diagnostic logging enriching internal-only detail fields
```

Anti-pattern spotlight

Returning raw sentinel strings upward unclassified—observer teams cannot categorize retry vs permanent failure cleanly.

Produce mini internal taxonomy—even if handwritten constants initially—staging future `errors.Is` adoption consciously.

Interview expectation: reconcile **explicit errors** bridging earlier fundamentals Area 01 with transport mapping discipline now.

Source: 05-http-apis-service-ownership-and-chi/09-http-timeouts-and-context-propagation.md

Unit 9 — Per-request timeouts & propagated context narrowing

Practice embedding synthetic slow payment collaborator sleeping beyond acceptable SLA enforcing earlier abort via tightened derived context timelines.

Demonstrate cooperating cancellation signals reaching nested repository fakers honouring `ctx.Done()`—even if repos still ephemeral memory until Area 08 deepens Postgres realism.

Enumerate strategies when legacy dependency wrappers lack native `context` signatures—adaptation façade sketch patterns minimally.

Interview drill: summarise failure classification when client disconnects prematurely mid-handler—observe possible wasted work nuances articulating politely.

Source: 05-http-apis-service-ownership-and-chi/10-service-repository-shape.md

Unit 10 — Service + repository seam rehearsal (thin handler religion)

Formalise layering narrative:

```
Handler (HTTP glue)
  → Application service orchestrating workflows + invariants bridging multiple repos eventually
    → Repository abstractions encapsulating persistence soon (temporary in-memory permissible learning stepping stone if documented transitional honesty)
```

Refactor drifting fat handlers surgically—even if simplistic domain rules currently.

Interview alignment previewing DDD-heavy Area 11 but pragmatic now: avoid cargo-cult aggregator services empty pass-through blindly—justify added layer cognitive tax via upcoming multi-repository coordination expectations honestly.

Source: 05-http-apis-service-ownership-and-chi/11-capstone-go-api-integration.md

Unit 11 — go-api/ capstone integration consolidation

Mandatory route surface practising integrated themes:

```
GET /orders
POST /orders
GET /products
GET /health
```

Stack obligations recap:

```
chi routing
explicit validation middleware / handler interplay
structured logging keyed by request/trace correlation fields
timeouts contextually aligned
explicit error envelopes / mapping strategy succinctly documented
middleware ordering rationale appendix short note
```

Deliverables (documentation-weighted—not code dump fetishisation)

Articulate succinctly:

ownership layering chain
anticipated failure taxonomy example table (even if partial)
latency budget sketch commentary at high-level
upgrade evolution hook (serialization/gRPC bridging forward Area06)

Interview consolidation checklist previously scattered—must verbally chain story uninterrupted simulating onboarding walk-through.

Phase — 06-grpc-protobuf-and-service-contracts

Directory: 06-grpc-protobuf-and-service-contracts/

Source: 06-grpc-protobuf-and-service-contracts/01-scope-protobuf-foundations.md

Unit 1 — Placement & protobuf linguistic foundations (`proto3` baseline)

Bridging cadence cue: dedicate ~4–6 sustained blocks AFTER HTTP ergonomics matured—ordering intentional.

Proto snippet orientation:

```
message User {  
  int64 id = 1;  
  string name = 2;  
}
```

Learning trajectory

Interpret **protobuf** contrasts JSON verbosity / dynamic typing ergonomics externally vs binary efficiency + schema coupling obligations internally.

Practice crafting evolving `order.proto` exercising:

messages • enums • repeated collections • cautious optional-ish fields respecting applicable proto version semantics precisely (verify toolchain docs!)

Maintain cross-language evolution empathy—even if downstream consumers hypothetical now.

Interview primer contrasts protobuf vs JSON not absolutism—articulate ambiguity debugging trade-offs honestly.

Source: 06-grpc-protobuf-and-service-contracts/02-unary-rpc-server-client.md

Unit 2 — Unary RPC choreography: codegen → server skeleton → exercised client harness

Practice service definition illustrative:

```
service OrderService {  
  rpc CreateOrder(CreateOrderRequest) returns (Order);  
  rpc GetOrder(GetOrderRequest) returns (Order);  
}
```

Produce Go server registration + minimal client exercised via CLI or integration test—even if ephemeral `bufconn` in-process style acceptable learning accelerant.

Enumerate contrasts REST multi-roundtrip chatter vs aggregated RPC shapes trade spectrum avoiding tribal absolutes.

Interview drill: verbally justify synchronous RPC chaining latency hazards previewing saga / asynchronous queue bridging forward areas.

Source: 06-grpc-protobuf-and-service-contracts/03-streaming-patterns.md

Unit 3 — Streaming modalities: selection discipline vs REST polling mimicry avoidance

Interpret streaming classifications:

```
server streaming notifications / changefeed-esque flows
client streaming batched ingestion without giant single payloads
bidirectional conversational patterns (often chat exemplar didactic—even if simplistic echo exercise suffices)
```

Practice restraint

Maintain intentionally **small purposeful streams**—not infinite toy loops starving learning closure.

Enumerate back-pressure conceptual interplay with grpc flow-control—high altitude explanation acceptable now.

Interview lens: articulate **when streaming genuinely justified vs REST chunked transfer / polling hybrids** thoughtfully—not streaming worship reflex.

Source: 06-grpc-protobuf-and-service-contracts/04-interceptors-metadata-deadlines.md

Unit 4 — Interceptors ↔ metadata ↔ deadlines (mirroring—but not cloning—HTTP middleware)

Implement cross-cutting auth header verification placeholder (explicitly insecure teaching stub—not production pattern misrepresented).

Practice propagating analogue **trace / request identifiers** bridging metadata headers cleanly.

Demonstrate layering deadlines (`context.WithTimeout` interplay) collapsing calls crossing configured budgets surfacing `codes.DeadLineExceeded` narrative mapping.

Interview nuance distinguishing unary interceptors chaining vs streaming `RecvMsg` interception complexity gradient awareness naming only minimally unless deeper dive chosen willingly.

Source: 06-grpc-protobuf-and-service-contracts/05-schema-evolution-compat.md

Unit 5 — Schema evolution & compatibility anxieties (most valuable long-run sub-skill)

Failure-class awareness

Dangerous impulses:

```
delete field numbering rashly rewriting history
reuse field numbers injecting silent mis-decoding nightmares
rename without dual-read transitional strategies in advanced production (preview conceptually—not mandatory full exercise now)
```

Reconciliation habits:

```
reserve retired numbers / names explicitly when removing
never recycle identifiers
paired deployment ordering sensitivity honesty
backward compatible additive changes preferred default
```

Lab writing deliverable short checklist

Enumerate safe evolution mini-playbook aligning potential organisational Buf / breaking change policies even if toolchain not mandated here—conceptual preparedness counts interview-wise.

Source: `06-grpc-protobuf-and-service-contracts/06-capstone-multi-service-grpc-flow.md`

Unit 6 — Multi-service gRPC choreography capstone sketch

Architect narrative—even if trimmed executable slice:

```
api façade service
payment microservice collaborator
inventory microservice collaborator communicating via grpc (local docker-compose optional—not mandatory if resource constrained articulate honestly)
```

Cross-cutting behaviours embed intentionally

timeouts • naive bounded retry scaffolding (justify caution overlapping idempotency area forward) • trace hook placeholders • propagated cancellation scenarios

Produce written sequence diagram-esque bullet timeline narrating synchronous chain fragility hotspots contrasted hypothetical asynchronous escalation patterns previewing Areas `12+/13`.

Interview expectation: cohesive story **without insisting finished production hardening artefacts** unrealistic at learning boundary—articulate deltas explicitly matured later.

Phase — 07-serialization-http-grpc-tradeoffs

Directory: 07-serialization-http-grpc-tradeoffs/

Source: 07-serialization-http-grpc-tradeoffs/01-json-protobuf-boundaries-internal-vs-external.md

Unit 1 — Serialization & protocol layering between REST(JSON) ⇔ gRPC(Protobuf)

Learning outcome

Articulate transport + encoding as **explicit architecture choices**—not accidental defaults drifting.

Comparative lenses

Concern	JSON / REST typical	Protobuf / gRPC typical
human debug agility	strengths	tooling mitigations (grpcurl /reflection hazards)
payload efficiency	variability	tendency strength
contract evolution coupling	swagger/OpenAPI ecosystem cultural momentum	codegen + breaking change vigilance intensified
browser friendliness	direct	gateways/bff patterns often necessary

Practice narrative vignette rewriting simplistic JSON REST synergy service pair into internal protobuf-backed RPC emphasizing **explicit compatibility contract maintenance tax** acceptance.

Interview focal phrases:

protobuf vs JSON trade matrix (no caricature extremes)
streaming-only-when-justified discipline
backward compatibility philosophies bridging HTTP OpenAPI ecosystems vs protobuf field numbering regimes
internal vs external gateway boundary clarity

This unit intentionally **dense prose** consolidating earlier HTTP + gRPC arc glue—reuse as rehearsal sheet before interviewing narratively bridging both worlds.

Phase — 08-persistence-sql-postgresql-and-sqlx

Directory: 08-persistence-sql-postgresql-and-sqlx/

Source: 08-persistence-sql-postgresql-and-sqlx/01-scope-persistence-capstone-order-system.md

Unit 1 — Persistence scope: Postgres + sqlx + migrations feeding go-api evolution

Informative cadence: approximate ten thematic blocks previously described at ~1-1.5 h/day—**sequence not calendar law.**

Learning outcome metamorphosis

Progress thinking from “I inserted a row” toward “**schema + transaction + concurrency semantics shape system behaviour over time.**”

Tooling alignment

Concern	Direction
RDBMS	PostgreSQL
DB access	sqlx explicit SQL ergonomics bridging stdlib database/sql
schema evolution	golang-migrate (equivalent disciplined tool acceptable—justify differences briefly if diverging)

Capstone foreshadow (**Unit 11**): **order-system/** practising realistic checkout lifecycle including forced failures sharpening reasoning—not happy-path façade only.

Source: 08-persistence-sql-postgresql-and-sqlx/02-relational-modeling-constraints.md

Unit 2 — Relational modelling constraints as domain guardrails (CREATE TABLE literacy)

Practice designing tables scaffold:

users • orders • payments • products (extend inventory flavour if desired)

Constraint literacy emphasis

Declare:

PRIMARY / FOREIGN KEYS mirroring relational integrity verbally defended
UNIQUE guarding business-natural identities thoughtfully
indexes anticipating realistic access paths (defer perfectionism paralysis—articulate iterative measurement culture)

Lab reflection

Articulate relational **ownership flows** aligning mental aggregate boundaries previewing architectural Area 11 without premature DDD dogmatism shackling schema iteration.

Interview prompts: cascading deletes cautions articulating nuanced business kill-switch policies vs DB enforcement sympathy.

Source: 08-persistence-sql-postgresql-and-sqlx/03-sqlx-query-patterns.md

Unit 3 — `sqlx` patterns: readability + scanning ergonomics resisting ORM concealment relapse

Operational API familiarity anchors:

```
Get / Select scoped queries
Named parameters / NamedExec structuring dynamic fragments cautiously guarded against injection regressions—even if placeholders mostly static early
mapping rows → structs thoughtfully (nullable fields handling strategies preview)
```

Practice repository scaffolding

Expose methods:

```
FindByID
Create
Delete (+ updates when flows demand)
```

Interview comparisons

Enumerate ORM pitfalls abstractly hiding N+1 query explosions / implicit eager loading vs explicit SQL readability tax honestly accepted.

Lab micro note: annotate query text readability strategies (capitalisation / line breaks)—reviewer empathy signalling professionalism early.

Source: 08-persistence-sql-postgresql-and-sqlx/04-repository-layer-ownership.md

Unit 4 — Repository ownership clarification (not dumping ground façade names)

Healthy responsibility slice:

Repositories encapsulate **persistence choreography mechanics** translating domain commands into SQL parametrisation—not arbitrary business branching logic accumulation swamps readability.

Thin application services orchestrating multi-repository stories remain authoritative location cross-aggregate rules live—preview expansions thoughtfully.

Practice refactor spotting **domain creeping** creeping inside repository methods prematurely—articulate corrective extraction movement narrative.

Interview drill: delineate `OrderService` vs `OrderRepository` interplay verbally crisp under synthetic scenario questioning cross-table invariant enforcement location.

Source: 08-persistence-sql-postgresql-and-sqlx/05-transactions-and-failure.md

Unit 5 — Transaction boundaries: coherent units of atomic commitment storytelling

Recall structural narrative:

```
BEGIN → business attempts → COMMIT triumph OR ROLLBACK failure restoration
```

Practice scenario deepening

Compose **checkout-esque** interplay:

```
payment ledger stub + inventory decrement + order finalisation statuses
```

Instrument intentional mid-flight failure provoking partial state danger absent transaction discipline—repair narratively emphasizing isolation expectations high-level—not serializable thesis unless advanced DB curriculum cross-linked elsewhere purposely.

Interview conceptual bridges toward distributed sagas (**Area 12**) juxtaposing truthful **non-existence mythical global transactions** bridging microservices contrast.

Source: 08-persistence-sql-postgresql-and-sqlx/06-locking-and-consistency-races.md

Unit 6 — Locking flavours & concurrency hazards (lost updates narrative)

Interpret **optimistic** (often version counters / conditional updates) versus **pessimistic** (`SELECT ... FOR UPDATE` conceptual awareness) interplay trade spectrum.

Practice contention scenario:

two simultaneous purchasers dwindling scarce inventory provoking race glimpses—implement guard strategy variant articulating anomaly prevention claims honestly referencing isolation knowledge bounds.

Interview nuance distinguishing application-level reconciliation vs leaning DB primitives consciously—ethical complexity admission encouraged vs hand-wavy “DB handles everything” mythology debunk politely.

Source: 08-persistence-sql-postgresql-and-sqlx/07-schema-migrations-operational-hygiene.md

Unit 7 — Migrations hygiene: repeatable up / down rehearsal discipline

Operational truth:

Production abhors improvised `ALTER` improvisations undocumented—migrate scripts reviewed like auth code guarding money.

Practice migration adding column illustrative:

```
orders.country (nullable rollout strategy pondering expand-contract glimpses bridging zero-downtime awareness external cross-program references optionally)
```

Rehearse downgrade path—even if ephemeral dev DB nuking acceptable—you still author reversible intent expressing operational seriousness.

Interview prompts: articulate coordination with application dual-read/write phases when uncompromisable uptime constraints realistic—defer deep pattern mastery referencing external dedicated units if enriched elsewhere purposely.

Source: 08-persistence-sql-postgresql-and-sqlx/08-query-ownership-dashboard-reads.md

Unit 8 — Query ownership readability: resisting SELECT * convenience rot

Articulate dashboards / analytics queries maintaining **explicit column projections** safeguarding forward schema evolution breakage detection earlier.

Practice constructing representative aggregate queries powering operational insights— revenue snapshot & top orders illustrative—even if seeded fixture data artificially small.

Enumerate observability interplay hooking hypothetical slow-query log inspection mindset bridging Ops program cross-links optionally.

Interview dimension: distinguishing **operational querying** burdens vs transactional hot path query tuning priorities balancing honestly.

Source: 08-persistence-sql-postgresql-and-sqlx/09-connection-pool-tuning-awareness.md

Unit 9 — Connection pool configuration awareness (database/sql tuning mental hooks)

Tune cognitively—even if qualitative:

```
SetMaxOpenConns
SetMaxIdleConns
ConnMaxLifetime
```

Relate bursts of concurrent handlers + goroutines to pool starvation vs Postgres resource exhaustion interplay narrative—not numeric perfection yet.

Interview expectation: verbally distinguish misblaming DB vs mistaken pool undersizing hypotheses methodically narrowing production incidents hypothetical walkthrough style.

Source: 08-persistence-sql-postgresql-and-sqlx/10-handler-service-repository-persistence-shape.md

Unit 10 — Layered persistence integration recap (transport → service → repository → DB)

Formalise repeatable chain aligning earlier HTTP layering Area 05 with matured persistence artefacts—prevent routing SQL leaking upward regressions creeping during deadline crunch fictions ethically acknowledged.

Conduct deliberate refactor extracting stray raw queries embedded historically—document mechanical steps + detected smell indicators future you recognises faster.

Interview consolidation mini drill: trace single CreateOrder request path enumerating failure surfaces per hop without peeking notes—muscle memory rehearsal target.

Source: 08-persistence-sql-postgresql-and-sqlx/11-capstone-order-system-integration.md

Unit 11 — order-system/ persistence capstone stitching failures intentionally

Mandatory lifecycle coverage sketch:

```
create order
pay order
cancel order
list orders (+ filtered variants optional)
```

Fault injection rehearsals (document recovery reasoning)

Craft scenarios:

transaction rollback correctness after nested failure invocation
locking race resurfacing interplay lessons unit6 repairs verification
purposefully malformed / conflicting migration practising detection + rollback choreography reverting
safely

Interview checklist aggregator

transactions • repository delineation • migration strategy • locking consistency • rollback narratives • pools •
sqlx explicit query ownership • interplay with architectural boundaries forward Area 11

Deliverable emphasises retrospective write-ups—“**why remediation structurally healed class of defect**”—
not ephemeral hotfix mysticism anecdotes.

Phase — 09-testing-engineering-go-services

Directory: `09-testing-engineering-go-services/`

Source: `09-testing-engineering-go-services/01-table-driven-unit-tests.md`

Unit 1 — Table-driven unit testing idioms sharpening Go clarity

Goals:

compose compact **cases slice** iterating subtests asserting behaviours systematically—not copy/paste divergence decay.

Articulate parallelism cautions (`t.Parallel`) respecting shared mutable fixture hazards unless isolated deliberately.

Discuss subtest naming strategies easing failure triage readability.

Interview contrast vs xUnit parameterized theories cultures briefly—focus pragmatic Go ergonomics—not framework wars trivia.

Source: `09-testing-engineering-go-services/02-integration-tests-and-testcontainers.md`

Unit 2 — Integration tests crossing process boundaries responsibly (containers optional realism)

Leverage ephemeral DB containers (**Testcontainers** pattern) validating migrations apply + repository queries behaving realistically—even if abbreviated pipeline acceptable learning scale document environmental constraints honesty if skipping heavy infra.

Enumerate teardown hygiene guarding leftover resources leaking CI minutes.

Discuss flakiness triage playbook categorising nondeterministic timing failures versus genuine defect signals—intellectual honesty training interview narrative polish.

Source: `09-testing-engineering-go-services/03-gomock-and-interface-seams.md`

Unit 3 — Interfaces enabling focused doubles (`gomock` or minimal hand mocks)

Practice isolating **OrderRepository** collaborator via mocks verifying orchestrating service behaviour—not SQL mechanics simultaneously—layering assertion clarity thoughtfully.

Enumerate mock over-proliferation pathology creating brittle suites mirroring implementation not contract—articulate heuristic mock boundaries verbally.

Bridging concurrency tests interplay awareness minimal mention—defer heavy synergy unless curiosity drives optional stretch.

Source: `09-testing-engineering-go-services/04-benchmarks-load-and-fuzz-hooks.md`

Unit 4 — Benchmark framing + exploratory load runs + fuzzing awareness groundwork

Operational commands literacy:

```
go test -bench
optional lightweight k6 scripted spike if curious—not mandatory core now
fuzz targets via go test fuzz harness for parsers / serializers prone edge combinatorics
```

Interview expectation: articulate interpreting noisy micro-benchmark deltas cautiously distinguishing signal vs ephemeral CPU frequency thermal noise honesty.

Discuss load testing aligning realistic workload modelling vs hello-world unrealistic QPS vanities debunk politely.

Source: `09-testing-engineering-go-services/05-capstone-coverage-balanced-suite.md`

Unit 5 — Capstone: pragmatic coverage pyramid aligning REST-ish service artefacts

Synthetic target heuristic from original authoring brief:

```
strive balanced >80% coverage on critical packages if metrics meaningful—not gaming via trivial asserts
paired integration slices exercising repository + transactional boundaries hot paths
minimal benchmark artefacts anchoring hotspots candidates future optimisation—not premature micro-optimisation fetish formation
```

Deliverable artefact expectation

Produce **Testing Strategy Markdown** succinctly aligning CI gates you'd propose defensibly onboarding team hypothetically—even if pipelines not enforced literally now.

Enumerate metrics anti-patterns (coverage mandates encouraging nonsense tests) guarding intellectual integrity politely.

Phase — 10-caching-redis-and-invalidations

Directory: `10-caching-redis-and-invalidations/`

Source: `10-caching-redis-and-invalidations/01-redis-cache-strategies-enterprise-mindset.md`

Unit 1 — Redis-assisted caching motifs & infamous invalidation dragons (enterprise realism)

Learning outcome depth emphasised original brief for **European enterprise-heavy hiring contexts**:

Strategy catalogue practising articulation—not production cluster mandatory immediately

cache-aside (lazy population)
write-through coherence emphasis variants
write-behind buffering latency wins + durability risk premium honesty
explicit TTL budgeting stories
stampede / thundering herd mitigation instincts (locking / probabilistic early expiration / singleflight layering conceptually—even if libs deferred)
distributed invalidation signalling complexity glimpses without Kafka detour mandates here unless curious optional

Practice architectural narrative:

Hot product catalogue read API backed by Postgres origin-of-truth
Redis cushioning read spikes conscientiously debating staleness envelope risks business stakeholders hypothetically confronting
explicit invalidation triggers on authoritative writes—not magical eventual consistency pretending precision

Interview crucible preparedness

Historical aphorisms worth owning fluently—not buzzword karaoke:

“There are **two hard things**...” finish thoughtfully articulating nuanced partial truths beyond meme repetition shallowly—demonstrate applied scenario nuance interviewer expects seniority signal.

Bridging observability implication: articulate metrics differentiating hits/misses/latency shifts detecting misconfiguration regressions responsibly.

Phase — 11-backend-architecture-ownership-and-design

Directory: 11-backend-architecture-ownership-and-design/

Source: 11-backend-architecture-ownership-and-design/01-scope-architecture-capstone-ecommerce-backend.md

Unit 1 — Architecture scope (ecommerce-backend/ mental spine)

Informative pacing anchor: historically ~12 deepening segments aligning ~1-1.5 h/day intent—ordering only.

Outcome refrain elevated:

advance from “**directory roulette layout anxiety**” to **articulate responsibility ownership crisply**:

DDD-flavoured thinking (avoid dogma theater)
CQRS literacy where read models diverge materially
manual dependency injection ergonomics resisting global state regression
explicit package boundaries guarding bounded contexts
conscious composition vs inheritance impulses relapse detection
bounded error narratives bridging earlier fundamentals with structural clarity upstream
clean-ish architectural layering pragmatic—not mythic circle generator cargo cult paralysis

Stack coherence cross-links deliberately

Leverage matured `chi` + `Postgres/sqlx` stack avoiding framework black-box magic contradictory learning objectives—compose explicitly.

Source: 11-backend-architecture-ownership-and-design/02-package-ownership-vs-util-soup.md

Unit 2 — Ownership primacy eradicating miscellaneous util/ entropy accumulation

Articulate malignant pattern:

Directories named `helpers`, `common`, `misc` metastasizing untyped dumping grounds thwarting cohesion reasoning during reviews prematurely exhausting cognitive budgets.

Prefer domain-centric packages aligning bounded language:

order
payment
inventory

Practice refactoring exercise

Fabricate purposely messy codebase snapshot—migrate toward coherent ownership slices documenting rationales resisting lazy re-exports.

Interview phrasebook entry: communicate **bounded context** narrative bridging DDD glossary later units intensifying interplay without buzzword bombardment devoid of substantive trade articulation.

Source: 11-backend-architecture-ownership-and-design/03-clean-architecture-layers-discipline.md

Unit 3 — Clean architecture layering instincts (transport → application → domain → infrastructure echoes)

Maintain directional dependency aspiration:

Outer layers adapting frameworks / DB drivers inward toward domain behavioural core minimizing outward leakage corrupting reusability instincts—Go pragmatism tolerates compromises when consciously documented contrasting absolutist paralysis.

Construct order flow layering diagram verbally reproducible interviewer prompts—handler thinness reiterated echoing Areas 05 & 08 matured earlier intentionally cross-referencing cohesion.

Lab nuance distinguishing architectural ideals vs iterative shipping pressures honestly—document conscious tech debt hotspots if temporarily bending purity with mitigation timers mentally scheduled.

Source: 11-backend-architecture-ownership-and-design/04-manual-di-and-compilation-safety.md

Unit 4 — Manual dependency injection clarity (compilation-safe wiring discipline)

Contrast anti-pattern caricature scattering `NewWhatever()` globals mutating incidental shared state covertly thwarting deterministic tests.

Prefer constructor injections accepting collaborators explicitly enabling substitution under tests without reflection magic frameworks typically unnecessary idiomatic Go.

Practice composing `OrderService` wiring repository + structured logger collaborator minimalistically—defer heavy DI frameworks purposely unless organisational policy mandates externally later verify reality then.

Interview drill: verbally justify manual wiring simplicity trade spectrum vs heavyweight runtime container introspection complexity costs rarely justified early lifecycles.

Source: 11-backend-architecture-ownership-and-design/05-domain-modelling-ddd-elements.md

Unit 5 — Domain vocabulary shaping: entities, value objects, aggregates (DDD-lite)

Differentiate stereotypes:

Entities identity lifecycle emphasis vs interchangeable immutable value snapshots with structural equality—not pointer identity narcissism unnecessarily.

Practice sketch **Order aggregate root** guarding invariants resisting anemic structs only nominally resembling domain—they must encode meaningful disallow transitions violating business storytelling minimal yet truthful.

Articulate interplay coupons / promotional rules / peripheral concerns intentionally sometimes **outside aggregate** guarding against mythical “do everything blobs” prematurely coupling unrelated churn rates.

Interview expectation: summarise **aggregate** vs stray entity divergence crisply verbally without textbook quotes regurgitating memorised definitions shallowly devoid of applicability anecdotes.

Source: `11-backend-architecture-ownership-and-design/06-aggregate-boundary-tradeoffs.md`

Unit 6 — Aggregate boundary tension interrogation sharpening trade coherence

Dangerous cohesion collapse archetype caricature lumping disjoint lifecycles—Orders + Inventory + Coupons + Reviews casually unified provoking oversized transactional gravity wells hindering iterative scaling.

Articulate narrower aggregate boundaries conscientiously—even if implying eventual conversational synchronisation choreography previewing CQRS saga interplay forward areas.

Produce trade-off memo confronting reviewer hypothetical challenging boundary tightness accusing over-segmentation—defend succinctly evidencing behavioural independence statistics qualitative if lacking quantitative backlog metrics early lifecycle honesty.

Source: `11-backend-architecture-ownership-and-design/07-cqrs-read-vs-write-shape.md`

Unit 7 — CQRS read vs write model divergence literacy (purposeful—not reflexive duplication)

Articulate caricature pitfalls:

Repositories attempting identical shapes satisfying divergent analytic read patterns producing accidental coupling throttling iterative schema evolution velocities.

Separate conceptual paths responsibly:

Command handling mutating aggregates persisting optimised write-centric schema shapes
Versus query surfaces projecting denormalised / dashboard-friendly aggregates consciously accepting duplication taxation only when evidenced necessity emerges—not speculative future-proof fetish premature rely.

Practice divergence sketch

Demonstrate differentiated read model powering dashboard metrics distinct relational shapes vs normalized transaction tables—articulate syncing strategy high-level glimpsing event-driven projections later bridging distributed area optionally—not mandatory exhaustive implementation fidelity now.

Interview nuance disclaimers emphasising CQRS operational complexity tax honesty counterbalanced scaling wins articulation precision anti-dogmatic balanced maturity signal.

Source: `11-backend-architecture-ownership-and-design/08-architectural-composition-recap.md`

Unit 8 — Architectural composition revisit (scaling mindset beyond micro snippet examples)

Elevate fundamentals composition motifs—embedding collaborators / assembling services orchestrating orthogonal capabilities—preventing creeping inheritance impulses resurrecting OO habits misaligned pragmatic Go codebase sustainability reviews.

Conduct cross-cutting collaborator audit enumerating duplication risks vs justified reuse boundaries balancing consciously.

Brief oral rehearsal contrasting PHP inheritance-heavy historical reflex vs Go assembling explicit small interfaces interplay bridging earlier foundational Area 01 spiral intentionally closing conceptual loop matured scale now.

Source: `11-backend-architecture-ownership-and-design/09-strategic-package-realignment.md`

Unit 9 — Strategic package refactors guarding compile-time cohesion signals

Conduct deliberate reorganisation manoeuvre improving discovery / import graph clarity—even if mechanically incremental diff emphasising readability over aesthetic folder naming artistry shallowly judged.

Enumerate risks large-scale moves incur merge conflict penalties—articulate phased migration strategies balancing reviewer fatigue vs incremental drift accumulation hazards truthfully nuanced.

Interview prompt: versioning internal shared modules hypothetical multi-team concurrency constraints glimpsing forward organisational scaling pressures beyond solo learning scope honesty encouraged.

Source: `11-backend-architecture-ownership-and-design/10-cross-boundary-integration-rules.md`

Unit 10 — Cross-boundary rule enforcement: domain purity vs pragmatic Go pragmatism balance

Danger examples:

Polluting pure domain structs with omnipresent JSON / DB tags collapsing separation advantages inadvertently—exceptions permissible when consciously enumerated with reversible migration thoughts scheduled.

Maintain converter layer discipline mapping domain ↔ persistence ↔ transport payloads intentionally—not accidental silent aliasing divergence corrupting behavioural guarantees silently.

Conduct audit pass identifying leaky boundary offenders proposing remediation slices prioritising review blast radius ethically.

Source: `11-backend-architecture-ownership-and-design/11-errors-as-architecture-signals.md`

Unit 11 — Architectural error narratives signalling deeper systemic health than status codes superficially flashing

Elevate naive `return err` sprawl critiques matured earlier layering explicit domain error taxonomies aligning observability classifications guiding operator triage distinctions previewing reliability engineering interplay optional cross-disciplinary bridging.

Articulate interplay between sentinel business conflicts vs infra transient outage classifications shaping retry policies thoughtfully bridging distributed Areas forward intentionally referencing hook words only minimally now.

Conduct mini matrix mapping error classes → corrective actions referencing hypothetical pager responsibilities maturity signal interviewers relish concise clarity.

Source: `11-backend-architecture-ownership-and-design/12-tradeoff-studies-hyperscale-hypothetical.md`

Unit 12 — Hyperscale hypothetical trade study (reasoning gymnastics without fake precision numerology)

Synthetic scenario authoring brief historically:

Imagine **≈5M monthly active-ish users** hypothetically & **≈100M orders** illustrative scale—articulate nuanced reasoning toggling optional accelerators:

CQRS divergence necessity? Read replica introduction vs caching amplification layering? Aggregate splits reconnaissance adjusting transactional contention hotspots? Operational cost sensitivities ethically acknowledged honesty vs buzzword binge shallow matrix spam lacking falsifiable claims humility signal mature hire trait.

Produce bullet trade matrix + explicit assumption ledger + falsification metrics you'd gather before rewriting architecture radically—intellectual humility prized.

Source: `11-backend-architecture-ownership-and-design/13-capstone-ecommerce-backend-integration.md`

Unit 13 — `ecommerce-backend/` architecture capstone integration consolidation

Mandatory functional arcs synthesising motifs:

```
order creation workflows
payments transitions + idempotency awareness bridging forward reminders
cancellation pathways reconciling inventory states consistently
dashboard read surfaces practising CQRS divergence earlier conceptual seeds if even lightweight specialised queries distinct shapes—demonstrating intentional duplication vs premature generalisation contrasts honestly
```

Documentation expectations emphasising WHY manifesto excerpts

Enumerate ownership rationale bullet clusters + boundary confrontation resolutions + conscientious shortcut admissions paired mitigation timers intellectual honesty signalling senior-like transparency balanced ambition vs pragmatism.

Oral rehearsal checklist aggregator bridging entire Area 11 interview lenses previously scattered—simulate uninterrupted guided tour narrative without peeking scaffolding notes muscle memory forging target.

Phase — 12-distributed-systems-resilience-and-messaging

Directory: `12-distributed-systems-resilience-and-messaging/`

Source: `12-distributed-systems-resilience-and-messaging/01-scope-payment-system-micro-slices.md`

Unit 1 — Distributed systems scope (`payment-system/` multi-binary narrative)

Suggested cadence cue: historically ~12 concentrated segments aligning ~1–1.5 h/day authoring intent—ordering only meaningful here.

Synthetic multi-service layout sketch—even if minimally executed locally conscientiously acknowledging scaling honesty:

```
api/  
payment/  
inventory/  
notification/  
queue/ (conceptual—even if logically embedded module early simplifying wiring)
```

Technology motifs emphasised responsibly

Leverage matured **Go + Postgres (+ Redis selectively)** interplay plus **practical RabbitMQ queue integration** practising operational semantics conscientiously—not merely echoing Kafka lore abstractly devoid of tactile acknowledgements—even if eventual Kafka deepening deferred purposely per original curriculum nuance bridging conceptual awareness only minimally unless curiosity extends independently.

Mindset transformational goal:

migrate thinking from “**dependency HTTP 200 fantasies everlasting**” to **engineering partial failure assimilation calmly**.

Source: `12-distributed-systems-resilience-and-messaging/02-partial-failure-reality-check.md`

Unit 2 — Partial failure worldview reset: scepticism toward transport success innocence

Operational axiom grounding:

Networking layers mislead—latency spikes, flaky TLS middleboxes caricature realities even if caricaturised verbally now—internalise never trusting ephemeral success glimpses blindly without bounded verification strategies articulating matured later units systematically.

Practice integrating synthetic randomised failure injector toggling flaky payment collaborator returning ambiguous errors interleaved successes forcing classification discipline previewing retries cautiously restrained forward units balancing storm hazards.

Interview drill prompt expecting narrative classification transient vs systematic vs caller misuse distinctions crisply verbally without confusion hand-waving indefinite “just retry blindly” regressions unethical hiring signal debunk politely.

Source: 12-distributed-systems-resilience-and-messaging/03-http-clients-timeout-mandates.md

Unit 3 — Outbound HTTP client timeout mandates (context-aware transports)

Contrast naive usages historically foot-gunning reliability:

Uncapped `http.Client` default behaviours lingering dangerously indefinite blocking goroutine pools
silently suffocating service capacity caricature vividly responsible interview narrative expectation.

Mandate aligning contexts / `http.Client.Timeout` interplay consciously document layered budgets previewing
cascading deadline tightening discipline matured earlier foundational contexts synergy intentionally closing
conceptual loop matured distributed framing now escalation scale.

Source: 12-distributed-systems-resilience-and-messaging/04-retry-policies-with-backoff.md

Unit 4 — Retry policies: backoff, jitter ceilings, disciplined caps

Anti-pattern caricature

A naive “retry loop until success” hammering an already degraded dependency deepens outages: the caller looks “helpful” while it amplifies load and hides the real SLA breach.

Responsible pattern skeleton

Implement a bounded retry helper (learning-scale is enough) with:

- a **maximum attempt count**,
- increasing delay between attempts (backoff shape you can justify),
- optional **jitter** to reduce thundering herds,
- clear classification: **retry only when duplication is safe or idempotent** (ties to Unit 5).

Interview narration

Articulate **retry storms** as a second-order incident and why operators care about cooperative client behaviour—not only server health.

Source: 12-distributed-systems-resilience-and-messaging/05-idempotency-keys-payments-double-charge-story.md

Unit 5 — Idempotency: why payments + timeouts + retries are dangerous without keys

Classic failure story:

charge initiated → client times out unsure → retries → duplicate financial side-effect nightmares

Learning outcomes

Treat **idempotency keys** (`request_id` / Stripe-style keyed requests, etc.—API-specific details belong in your docs, not folklore) as a design surface:

- deterministic dedupe keyed by caller intent scope,

- storage of accepted vs processed keys with explicit retention policy thinking,
- aligning HTTP status semantics with ambiguity (accepted vs succeeded).

Practice

Simulate duplicated submit attempts verifying only one outward charge effect occurs under your deliberately imperfect timeout policy.

Interview prompts

Distinction between *at-least-once delivery* honesty and pretending *exactly-once* globally.

Source: `12-distributed-systems-resilience-and-messaging/06-queue-decoupling-rabbitmq-basics.md`

Unit 6 — RabbitMQ queue decoupling: move work off the critical path ethically

Contrast overloaded synchronous fan-out caricature pushing email/analytics/notifications inline until checkout latency becomes user-facing hostage situation.

Responsible outline:

checkout critical path ⇒ enqueue durable intent message ⇒ worker draining asynchronously respecting consumers operational realities acknowledging new tail latency failure classes appearing honestly

Practice

Wire a modest publisher/consumer with explicit ack/nack reasoning—understand poison messages preview next DLQ discussions.

Interview expectation: verbally contrast queue introduction tax (operability, duplication handling) vs latency wins / blast-radius containment virtues balanced non-absolutism.

Source: `12-distributed-systems-resilience-and-messaging/07-delivery-semantics-at-most-once-least-once.md`

Unit 7 — Delivery semantics: at-most-once, at-least-once, and “exactly-once” marketing vs engineering honesty

Distributed messaging rarely matches marketing slogans. Your job is to state what your system actually provides and what handlers must tolerate.

Semantics sketch

At-most-once

A message might never arrive. You must not rely on implicit delivery unless the business accepts loss.

At-least-once

Duplicates are possible unless you dedupe/idempotently process. RabbitMQ workloads are commonly in this territory unless you add strict dedupe semantics elsewhere.

“Exactly-once”

Often shorthand for engineered outcomes (idempotent handlers + dedupe keys + transactional outbox patterns), not an OS/network primitive you get for free.

Practice

Replay the same queued job twice deliberately. Confirm your consumer does not corrupt state (`INSERT` blindly without keys, naive counters exploding, duplicate side effects).

Interview prompts

- What does your broker actually guarantee vs what your application must guarantee?
 - Where do you dedupe—in DB, metadata store, keyed business tables?
-

Source: `12-distributed-systems-resilience-and-messaging/08-dead-letter-queues-recovery-flows.md`

Unit 8 — Dead-letter queues: escape infinite poisonous retry traps

Operational story: a malformed message or a deterministic bug retries forever—burning CPU, harming shared brokers, burying legitimate traffic.

Define a humane policy:

- bounded retries while errors look transient / unknown,
- then route poison messages into a **dead-letter queue** path (conceptual name varies by broker) for isolated inspection/replay tooling.

Document what operators need: payloads, timestamps, correlation headers, replay procedure that does not blindly double-charge (link idempotency, Unit 5).

Interview prompts

Difference between DLQ as “junk drawer” versus an operational cockpit with alerting and replay runbooks tied to compensation logic.

Source: `12-distributed-systems-resilience-and-messaging/09-sagas-compensating-actions.md`

Unit 9 — Sagas & compensating actions across services

Across service boundaries there is usually **no** single ACID transaction that can atomically commit `payment + inventory + email + loyalty points`. Coordination becomes an explicit workflow with **partial failure** baked in.

Core mental model

- **Forward actions** advance business state (`ReserveInventory` , `CapturePayment`).
- **Compensating actions** undo earlier forward steps best-effort when later steps fail (`ReleaseReservation` , `RefundPayment`), subject to strict business/legal rules—not every rollback is symmetrical or instant.

Design tasks:

1. Decide what “done” means for each step and what artefacts prove it (records, statuses, immutable events).
2. Make steps **retry-safe** using idempotency keys where external systems can double-apply ambiguous calls.
3. Document where you cannot compensate fully (often payment rails); pair with ops runbooks honest about manual intervention.

Compare coordination styles

- **Orchestration:** central component drives next steps explicitly (think workflow engine vibes or your own saga coordinator)—clear flow, coupling risk concentrated.

- **Choreography:** services react to signals/events—fewer choke points but harder reasoning about implicit contracts.

Interview prompt: sketch one concrete failure midpoint and enumerate compensations invoked in order—and what happens if a compensation fails.

Source: `12-distributed-systems-resilience-and-messaging/10-backpressure-queue-growth-throttling.md`

Unit 10 — Backpressure when producers outrun consumers

Problems appear as **queue depth growth**, goroutine buildup, latency balloons, RAM pressure—or worse, masking failures until cascading timeouts occur.

Signals to watch mentally (even qualitatively in a toy):

```
ingress rate sustained > worker drain capability
timeouts rising while CPUs look partially idle => blocking / IO / lock contention hypotheses
producer goroutines spawning without bound drowning scheduler fairness illusions ethically debunk politely
```

Intervention patterns (pick consciously)

- Bounded queues + rejecting/shedding load at the HTTP edge when saturated.
- Throttling callers (tokens, leaky buckets) before they enqueue unbounded speculative work.
- Scaling workers realistically (more CPU-bound vs IO-bound distinctions matter).
- Revisiting idempotency and retry aggression when storms self-inflict outages.

Interview drill: articulate where backpressure is applied (client, gateway, publisher, broker, consumer batch sizing) and the user-visible degradation you accept under overload.

Source: `12-distributed-systems-resilience-and-messaging/11-distributed-tracing-correlation-notes.md`

Unit 11 — Correlation IDs and minimal distributed tracing awareness

Failures cross service hops; plaintext “error happened” logs at each hop recreate detective novels nobody finishes.

Practice baseline

Attach and propagate minimally:

```
request id (edge assigned)
upstream trace-ish correlation token (whatever your org standardises—don't Cargo-cult a vendor name here blindly)
tenant/user identifiers ethically cautioned PII sensitivities—even learning exercises practise redaction instincts
```

Log fields should line up grepably across hops even before full OpenTelemetry depth—establish mental habit: **carry context into nested calls.**

Interview prompts

Cardinality explosion when logging high-cardinality labels indiscriminately.

Sampling intuition: you cannot economically retain every span forever—articulate pragmatic retention vs debuggability tensions honestly.

Source: 12-distributed-systems-resilience-and-messaging/12-chaos-micro-lab-catalog.md

Unit 12 — Chaos rehearsal catalogue (small, controlled fault injections)

Build intentional failure rehearsals—notebook/diagram-heavy is fine—to complement code:

Fault	Symptoms you expect	Fix story you must articulate
payment backend slow / flaky	timeout storms, cascading cancellation	tightened budgets + retry sanity + degraded UX policy
queue broker restart	backlog spike, duplicated deliveries surfaced	durable publisher confirms / consumer idempotency / DLQ path
worker crash mid-batch	orphaned partial writes unless transactional boundaries exist	transactional outbox glimpses responsibly optional advanced bridging
network partition caricature verbally	ambiguous success	idempotency + reconciliation queries patterns

Produce a miniature **timeline write-up**: detection → containment → remediation → preventative guard next iteration.

Interview mindset: rehearsals beat heroics—you can describe how you practise partial failure calmly.

Source: 12-distributed-systems-resilience-and-messaging/13-capstone-payment-system-integration.md

Unit 13 — Capstone: resilient payment-system/ checkout choreography

Compose an end-to-end story (code depth proportionate honest to your sandbox):

```
checkout
  → payment
  → inventory reservation / adjustment choreography
  → queue-backed notification acknowledgement path
```

Must deliberately exercise earlier motifs

timeouts • bounded retries • idempotency keys • RabbitMQ-ish queue semantics pragmatically • delivery honesty • DLQ emergence • saga/compensation narration • correlation identifiers propagation minimally • backpressure narrative when workers insufficient capacity honesty

Fault replay notebook

Enumerate duplicate requests, ambiguous timeouts (“did payment land?”), poison messages, flaky workers—recover with written rationale bridging **delivery semantics**, **handler idempotency**, and **human-runbook** pointers where automation cannot ethically replace judgement.

Interview rehearsal: summarise the system aloud as if onboarding a sceptical teammate—prioritise correctness arguments over toolchain trivia.

Phase — 13-workflow-durable-execution-and-temporal-concepts

Directory: 13-workflow-durable-execution-and-temporal-concepts/

Source: 13-workflow-durable-execution-and-temporal-concepts/01-durable-workflows-temporal-shape.md

Unit 1 — Durable workflows & Temporal-style engines (conceptual)

Bridging cadence cue: about three-four thematic blocks once distributed motifs feel grounded—numbering is topic order, not a timetable.

Queues and hand-rolled sagas handle plenty of workloads. They become painful when work is **long-running**, crosses **slow humans or partners**, accumulates complex **retry + compensation graphs**, or must survive **process crashes** without losing deterministic progress bookkeeping.

Engines such as **Temporal** (conceptual label here—confirm vendor docs/SLO realities before organisational adoption commitments) elevate ideas like:

workflow = durable state machine coordinating activities
activities = retries/timeout/compensation wrappers around side-effects
replayable histories reconstructing unfinished work after outages

Positioning prompts (write answers, not trivia)

Contrast three recovery owners for staged failures (payment authorised , inventory commit fails shipping prep , cancellation requested mid-flight) under:

Approach	Strengths	Pain you accept
Bounded queue choreography	simplicity, clear operational story	brittle state bookkeeping if durations stretch
Hand-rolled saga/orchestration	maximal control code-side	sprawling logic + divergence across teams unless disciplined
Durable workflow engine	explicit timelines + deterministic replay story	infra/operational sophistication + deterministic workflow code obligations

Add:

1. A short **sequence sketch** for payment → inventory → shipping , with inventory failing and a compensation/refund arc.
2. For each ambiguity class (**timeout unsure**, **duplicate retry**, **late success after cancel**), name the **stored source of truth** (table/state/history/event) your design relies on—as you would defend it in review.
3. One closing paragraph naming when a workflow engine’s **replay/determinism** obligations outweigh the operational overhead, versus staying with queues + explicit sagas.

Phase — 14-messaging-advanced-kafka-and-event-ownership

Directory: 14-messaging-advanced-kafka-and-event-ownership/

Source: 14-messaging-advanced-kafka-and-event-ownership/01-scope-from-queue-to-log-event-backbone.md

Unit 1 — From “we have a queue” to durable event backbone thinking

You already practise **broker-backed work decoupling** (RabbitMQ-style paths in Area 12). This area stretches into **Kafka-shaped event streaming** semantics and **ownership patterns** (outbox/inbox) that staff engineers defend in payment and inventory pipelines.

Outcomes

- Place **Kafka** conceptually alongside RabbitMQ—not “Kafka everywhere,” but knowing its partition + consumer-group + ordering guarantees story.
- Name **replay**, **ordering**, **poison** handling, **event-driven boundaries** crisply—not only happy-path enqueue/dequeue instincts.

Suggested cadence (informational only): this area often lands as several deep blocks (~1–1.5 h/day authoring pace)—folder numbering is ordering, not a calendar.

Practice spine (conceptual sketch)

Maintain the narrative **payment → event emission → inventory**, where **event** carries explicit contract versioning and deterministic consumer behaviour under duplicates.

Interview cornerstone

Articulate **RabbitMQ vs Kafka** responsibilities: workload shape, throughput, replay model, retention, ordering primitives, ops surface—reject absolutist slogans.

Source: 14-messaging-advanced-kafka-and-event-ownership/02-kafka-foundations-partitions-and-ordering.md

Unit 2 — Kafka conceptually: topics, partitions, offsets, replay

Kafka is marketed as “log,” behaved like **append-only segments** with scalable fan-out—not a mailbox that deletes after acknowledgement.

Core vocabulary you must wield:

Concept	Operational meaning
Topic	named stream grouping related events (<code>payment.events.v1</code>).
Partition	unit of parallelism; ordering is partition-scoped , not globally magic.
Offset	per-partition consumption progress pointer; rewind/replay rewinds offsets intentionally.
Retention	events may remain readable historically—replay becomes a deliberate tool, also a GDPR/PII headache if ignored.

Ordering reality

Kafka gives **total order inside one partition**. Across partitions ordering is meaningless—never assume global ordering without pinning keys.

Practice question: **PaymentCaptured ordering vs InventoryReserved causal expectations** — which saga step must serialize on the same partitioning key?

Lab

Compose two alternate partition key schemes for `payment` → `inventory` choreography and critique **skew** (single hot key) versus **incoherent ordering**.

Interview focus

Contrast “**message queue delivery**” folklore versus **replayable log semantics** succinctly—not vendor trivia.

Source: `14-messaging-advanced-kafka-and-event-ownership/03-consumer-groups-rebalancing-commit-semantics.md`

Unit 3 — Consumer groups & partition assignment

Kafka **consumer groups** coordinate who reads partitions:

```
group.members assign partitions ⇒ each partition processed ~once per consumer in steady state
(cooperative concurrency model)
```

Learning anchors

Understand **partition rebalance storms** briefly: scaling consumers or rolling deploys reshuffles assignments—paired with **incorrect offset commit strategies** duplicates or gaps amplify.

Enumerate **delivery honesty** interplay:

- **at-least-once processing** paired with dedupe/idempotency (see inbox pattern next area block),
- “exactly-once Kafka processing” myths vs engineering compensations realistically.

Practice reflection: correlate **Lag metrics** intuition—what infra alarms should exist even if dashboards stub learning-level only now.

Interview drill: summarise **scaling consumer count** ramifications on ordering & hot partitions verbally.

Source: `14-messaging-advanced-kafka-and-event-ownership/04-outbox-pattern-reliable-cross-service-publication.md`

Unit 4 — Outbox pattern: never “successful DB txn + forgotten publish”

Problem class: transactional success commits but message publish crashes—downstream ignorance permanent.

Canonical pattern

Persist **desired publication records** atomically beside domain writes (same DB transaction ideally), asynchronously relayed safely to Kafka/Rabbit with **publisher confirm / retry / DLQ bridging** interplay.

Articulate transactional boundaries honestly:

```
BEGIN
  mutate domain aggregates
  INSERT outbox_events (payload, routing metadata, ...)
COMMIT
```

background relay reads outbox → publishes → marks dispatched (or transactional delete variant—design choice with trade-offs)

Practice angle

Implement minimal table + relay loop teaching—even SQLite acceptable—contrasting transactional honesty vs heroic “dual write” folklore.

Interview prompt: reconcile outbox duplication risk & idempotent publisher behaviour.

Source:

[14-messaging-advanced-kafka-and-event-ownership/05-inbox-pattern-idempotent-consumer-safety.md](#)

Unit 5 — Inbox pattern: consumers survive duplicates & replays elegantly

Symmetric consumer-side companion to duplicates arriving under **>=1 delivery**:

Record message IDs consumed / processed transitions idempotently to avoid doubling side effects adjusting inventory dangerously.

Minimal mental model sketch:

```
BEGIN
  INSERT inbox_seen (consumer_group, producer_event_id UNIQUE)
  IF conflict → ack & skip gracefully
ELSE
  apply domain mutations
COMMIT
```

Articulate distinctions:

- deterministic idempotency keys vs brute dedupe ledger,
- long retention vs compaction nuances high-level—not broker admin depth requirement here.

Interview lens: summarise **difference outbox publishes vs inbox dedupes ingestion** verbally crisp staff expectation.

Source: [14-messaging-advanced-kafka-and-event-ownership/06-poison-messages-dead-letter-and-replay-governance.md](#)

Unit 6 — Poison scenarios, replay, operational governance—not endless retry loops

Poison message caricature deterministic bug or schema mismatch causing infinite catastrophic failure churn—must graduate to segregated remediation path analogous DLQ philosophies yet broker-specific knobs differ.

Operational responsibilities:

```
bounded retries distinguishing transient infra vs poisonous semantic failure
replay tooling auditable respecting idempotency + legal duplication constraints consciously
prevent unsafe mass replays multiplying financial inconsistencies responsibly
```

Kafka nuance teaser

Understand **offsets commit** interplay with poisonous partition stall—whole partition stuck if single bad record mishandled—contrasts naive queue acknowledging pattern assumptions.

Interview scenario: formulate policy table classifying retries vs skipping vs patching schema vs deploying fix-forward strategies.

Source: 14-messaging-advanced-kafka-and-event-ownership/07-event-driven-ownership-contracts-evolution-and-sagas-bridge.md

Unit 7 — Event-driven ownership: contracts, versioning, choreography discipline

Elevate storytelling beyond payloads:

Ownership concern	Behavioural question
Schema/version	Breaking vs additive evolution & dual publishing windows.
Causal boundaries	Which events must imply prior state guarantees?
Compensation interplay	How inventory corrective events interplay with saga failure arcs from Area 12 .
Trace correlation	Propagate linkage IDs bridging Area 06 / 12 motifs intentionally.

Event-driven temptation anti-pattern: anarchic choreography without explicit **ownership matrix** collapsing debugging into distributed guesswork responsibly debunk verbally.

Produce mini **ownership table** hypothetical `payment.capture` , `inventory.reserve` , `inventory.compensated` delineating emitting team & consuming invariants verbally articulable interviewer satisfying clarity signal matured deliberately.

Source: 14-messaging-advanced-kafka-and-event-ownership/08-capstone-payment-event-inventory-narrative-Tab.md

Unit 8 — Capstone narration: payment → verified event emission → inventory reaction

Produce written architecture & sequence justification (diagram optional) practising earlier motifs—not necessarily full infra cluster fidelity now.

Mandatory reasoning hooks (tick each explicitly in prose):

transactional persistence + outbox (or explicit refusal with compensating design + risks stated)
partitioning key coherence vs hotspots / skew honesty
inbox / idempotent consumer interplay OR alternative dedupe discipline stated
bounded retries vs poison isolation + replay governance story
Lag / backlog ageing metrics intuition (even if dashboards are stubs)
RabbitMQ vs Kafka trade matrix from your Areas 12 ↔ 14 story-no slogan absolutism

Deliver **short design note** citing:

Section	Requirement
Data path	Payment commit → emission → inventory acknowledgement.
Failure modes	Poison, duplication, partitioning skew, saga compensation tie-in at least verbally.
Interview compare	Structured bullet RabbitMQ vs Kafka without absolutism—state operational triggers choosing each.

Oral rehearsal: summarise end-to-end in ≤3 minutes calmly staff-style.

Phase — 15-performance-profiling-and-perf-lab

Directory: 15-performance-profiling-and-perf-lab/

Source: 15-performance-profiling-and-perf-lab/01-scope-latency-ownership-measure-first.md

Unit 1 — Performance mindset shift for perf-lab/

Informative cadence: historically ~twelve deepening blocks aligning ~1–1.5 h/day authoring intent—**folder order only**.

Reject vague “slow” emotions. Acquire **measurable evidence** distinguishing:

CPU hotspots vs allocation pressure/GC chatter vs contention vs IO/database/network culprits

Spine codebase: **perf-lab/** housing micro workloads you benchmark & profile—not production microservices yet.

Signature tooling internalisation:

```
go test -bench=. / select benches
go test -bench=. -benchmem
go tool pprof (CPU / heap endpoints or profile files)
net/http/pprof hooking practise eventually (next units)
```

Source: 15-performance-profiling-and-perf-lab/02-benchmark-thinking-and-microbenchmark-hygiene.md

Unit 2 — Benchmark discipline: interpreting go test -bench

Outcomes

- Write meaningful benchmarks (**string parse variants**, **encoding/json** decode workloads, **map** lookup micro shapes).
- Compare **Implementations A/B** without cargo-cult B/op fantasies—you still reason about compiler optimisations / dead-store elimination pitfalls.

Starter patterns:

```
go test -bench=.
go test -bench=. -benchmem
go test -bench=. -benchtime=... (when smoothing noise conscientiously—avoid benchmarking forever chasin
g ghosts)
```

Lab

Pick two parsing strategies (regex vs hand-rolled tokenizer mini, or **strconv** variants) and document **which metric moved** (ns/op, allocs/op) and **why** you trust the comparison setup (same CPU governor awareness high-level okay).

Interview prompts

Micro-benchmark traps: compiler eliding work, unrealistic tiny inputs, no **-benchmem** blind spots.

Source: 15-performance-profiling-and-perf-lab/03-allocation-pressure-stack-heap-first-pass.md

Unit 3 — Allocation thinking: pressure, escapes, observable growth

Loops that allocate per iteration pressure **GC**—sometimes dominating wall time more than naive Big-O guesses.

Practice:

Craft **append** / string concatenation pitfalls vs efficient patterns—observe **allocation count** deltas using `benchmem`.

Relate intuitively:

Question	Probe
stack vs heap	not always predictable without tooling—use profiler + escapes where confused
value copying	slicing big structs unknowingly allocates / copies bridging earlier fundamentals

Lab essay

“When allocation rate rises disproportionately versus CPU hotspots” diagnostic steps—mention `go tool pprof` heap sample preview forward units.

Interview focus: articulate **difference algorithmic complexity vs allocator chatter** verbally crisp.

Source: 15-performance-profiling-and-perf-lab/04-cpu-profiling-http-pprof-workflow.md

Unit 4 — CPU profiling harness (`net/http/pprof` + CLI workflow)

Practice enabling:

```
import _ "net/http/pprof"
```

(or bench-only programmatic `pprof.StartCPUProfile` capturing)

Generate artefacts:

```
go tool pprof -http=:6060 cpu.prof // or textual top / list commands
```

Exercise

Fabricate knowingly slow algorithm (naive substring scanning, quadratic loops) verifying dominant functions appear unmistakably in **CPU profile**.

Deliverable narration

Produce short **pseudo flame-graph interpretation paragraph** bridging textual `top`, `list`, optionally interactive graph.

Interview drill: summarise reading CPU graphs contrast heap profiles upcoming unit differentiate.

Source: 15-performance-profiling-and-perf-lab/05-heap-profiling-live-vs-allocated-objects.md

Unit 5 — Heap profiling growth hunting

Induce purposeful **steady memory ascent** caching slices incorrectly / retaining unintended references / global hoarding—even learning-scale acceptable.

Observe:

```
go tool pprof -http=:6060 heap.prof // focussing alloc_space vs inuse_* semantics consciously
```

Lab differentiation

Clarify in writing:

Profile view	Typical question
alloc_space / alloc_objects	“What allocates a lot—even if transient?”
inuse_space / inuse_objects	“What stays alive unexpectedly?”

Interview drill: differentiate **profiler evidence** from guesses when someone says “memory leak”—tie to goroutine/leak drills next units.

Source: 15-performance-profiling-and-perf-lab/06-garbage-collector-allocation-pressure-and-pauses.md

Unit 6 — Garbage collector thinking: allocation pressure & pause awareness

Go’s GC is **non-magical**: more allocation churn generally means more GC work (CPU + latency variance), even if individual pauses are often short relative to classic stop-the-world horror stories.

Practice

Allocate ~100k short-lived objects in tight loops under bench harness—watch **GC-related cost** indirectly via:

```
-benchmem  
cpu + heap profiles (previous units)  
optional GODEBUG=gctrace=1 textual logs (interpret qualitatively; don’t overfit local laptop numbers)
```

Lab

Explain in prose what **allocation pressure** means for tail latency at API scale—not average-only dashboards delusions.

Interview prompts

Generational GC myths misapplied to Go—stay factual with current runtime docs when disputing trivia.

Trade-off: micro-alloc elimination vs readability—optimise only with evidence (Area intent).

Source: 15-performance-profiling-and-perf-lab/07-memory-leak-patterns-goroutines-slices-channels.md

Unit 7 — Memory “leak” thinking in Go: goroutines, slices, channels

Go can **retain memory** without classic malloc/free leaks:

Pattern	Symptom sketch
Goroutine leak	forgotten <code>ctx</code> cancellation / blocked sends / waiting forever
Slice backing array retention	tiny visible slice holding giant underlying array alive
Channel / map caches	unbounded growth “feature”

Practice

Seed an intentional **goroutine leak** (ignore `Done` + blocking channel), observe process RSS climb / goroutine count via `pprof` goroutine profile or runtime metrics patterns.

Fix with cancellation + join discipline cross-linking Area 03/04 .

Interview expectation

Differentiate **unreachable cycles** (GC reclaim) vs **reachable accidental retention** (profiler shows live objects still referenced).

Source: 15-performance-profiling-and-perf-lab/08-slice-and-map-performance-patterns.md

Unit 8 — Slice & map performance: capacity, growth, bucket physics (high level)

Slice preallocation:

```
s := make([]T, 0, 1000) // fewer reallocations vs blind append growth
```

Maps: growth & rehash events exist—avoid hot loops recreating maps unintentionally; pre-size when approximate cardinality known.

Lab

Benchmark **prealloc vs naive append** and a **map heavy** micro workload; narrate `B/op` + `allocs/op` shifts.

Interview prompts

When preallocation misleads (overalloc memory permanently for tiny inputs).

Map iteration order non-determinism unrelated but often adjacent confusion—don't conflate with perf story unless relevant.

Source: 15-performance-profiling-and-perf-lab/09-concurrency-performance-and-worker-scaling-laws.md

Unit 9 — Concurrency performance: more goroutines ≠ faster

Measure:

```
worker pool sizes: 5 vs 50 vs 500
```

on **CPU-bound** vs **IO-bound** toy workloads—observe contention, scheduling overhead, and throughput collapse when oversubscribed CPU work.

Lab deliverable

Plot a small qualitative table of outcomes (even handwritten) and explain how they relate to coarse parallel speedup intuition (CPU-bound saturation vs IO-bound overlap)—without treating a laptop benchmark like a mathematical proof.

Source: 15-performance-profiling-and-perf-lab/10-json-and-serialization-hot-path-benchmarks.md

Unit 10 — JSON / hot-path serialization performance benching (encoding/json first)

Bench:

small structs vs medium vs large payloads
Decoder reuse vs naive patterns where applicable thoughtfully
alternative codecs only as optional appendix if curious—maintain honesty verifying semantics compatibility trade-offs consciously

Lab

Document where JSON becomes bottleneck vs database/network overshadowing guesses—articulate investigative question ordering for “slow endpoints” bridging next latency ownership unit.
Interview focus: schema shape & field count ripple effects—not just “JSON slow” slogans.

Source: 15-performance-profiling-and-perf-lab/11-latency-ownership-and-bottleneck-triage.md

Unit 11 — Latency ownership: triage CPU / memory / GC / DB / network systematically

Synthetic production scenario:
You receive “this API is slow.” Forbidden answer: random micro-optimisation gut feelings.

Checklist mental walkthrough

Layer	Quick honest probes
CPU	profile hot functions
Alloc/GC	-benchmem , heap profile, alloc pressure
DB	query plans / connection pool waits (cross Area 08)
Network	timeouts, retries storm, payload bloat
Queues	backlog age / consumer lag (Areas 12 / 14)

Practice

Introduce intentional slow dependency (sleeping fake client) AND independent CPU hog path—prove you can separate symptoms using tools rather than story-telling.

Deliverable

Short **incident note template**: symptom → hypothesis → validation evidence → fix → trade-off.

Source: 15-performance-profiling-and-perf-lab/12-profiling-incident-synthetic-lab.md

Unit 12 — Profiling incident lab (synthetic): CPU pegged, memory climbing, workers stalling

Compose multi-signal chaos in perf-lab/ :

CPU ~95% busy spin or pathological algorithm
memory growth via leak pattern(s)
worker pool starved or oversubscribed scenario mixing IO + CPU skew

You must:

profile (CPU + heap at minimum)
identify dominant contributors
patch with measured before/after bench/prof snapshot commentary
document trade-offs (readability loss? bounded buffers?)

Interview rehearsal: narrate investigation without opening laptop—prove muscle memory.

Source: 15-performance-profiling-and-perf-lab/13-capstone-payment-worker-performance-lab.md

Unit 13 — Capstone: `payment-worker/` performance integration

Build (or extend) `payment-worker/` exercising:

worker pool + queue consumption + retry + timeout discipline (bridge Areas 12/14 concepts lightly)

Then inject **deliberate pathologies**:

allocation / GC pressure scenario
goroutine leak scenario
memory retention via slice backing array trick (educational)

Use `pprof` + **benchmarks** to detect and fix—document **WHY** fixes work structurally.

Interview consolidation checklist (must speak smoothly):

`pprof` CPU/heap
benchmark discipline
GC / allocation pressure intuition
leak classes
slice prealloc trade-offs
stack vs heap first-pass literacy (deepening Area 16/23)

Phase — 16-go-runtime-internals-foundations-for-profilers

Directory: `16-go-runtime-internals-foundations-for-profilers/`

Source: `16-go-runtime-internals-foundations-for-profilers/01-gmp-scheduler-model-and-blocking-intuition.md`

Unit 1 — Scheduler model (G-M-P) for purposeful performance reasoning

Interpret **G** goroutine descriptors, **M** OS-thread workers, **P** logical processors/context carrying runnable queues—high level enough for interview storytelling without pretending you'll hand-derive preempt points daily.

Linkage to earlier work:

Why “more goroutines” thrashes CPU-heavy tasks (Area 15 Unit 9) maps to runnable scheduling contention & **P** starvation intuition.

Practice: summarise **blocking syscalls**, **park/unpark**, and **work stealing** verbally from current reputable runtime docs—not legacy blog myths blindly.

Source: `16-go-runtime-internals-foundations-for-profilers/02-stacks-heaps-and-escape-analysis-instincts.md`

Unit 2 — Memory model basics: stacks, heaps, escape analysis instincts

Understand **why** escapes matter economically (heap allocation + GC pressure) even if correctness remains fine.

Operational bridge:

```
go build -gcflags="-m" (escape diagnostics noise—interpret cautiously—compiler updates shift lines)
benchmark + heap profile evidence still king when confused
```

Interview prompt

Misusing escape logs as astrology—profiler remains authority for hot paths realistically.

Source: `16-go-runtime-internals-foundations-for-profilers/03-gc-basics-under-allocation-pressure.md`

Unit 3 — Garbage collection under the profiling lens

Connect GOGC semantics at high altitude (defaults, throughput vs RSS trade knobs) strictly as **experiment with measurement**—not production tuning folklore from one laptop sample.

Practice reading **allocation-driven GC overhead** interplay using `pprof` + optional `gctrace` qualitatively.

Cross-link Areas 15 / 23 for deeper dives—avoid duplicating false-sharing micro-arch here unless bridging sentences only.

Source:

16-go-runtime-internals-foundations-for-profilers/04-execution-trace-overview-go-tool-trace.md

Unit 4 — Execution trace alongside `pprof` (when wall-clock latency hides CPU-only blind spots)

Use `go tool trace` over a bounded window when:

- CPU profiles look fine but latency tails are bad (scheduling/blocking blindness),
- you suspect goroutine coordination or syscall waits dominate wall time.

Traces grow large quickly—capture short, representative workloads and delete artefacts you do not need.

Source: 16-go-runtime-internals-foundations-for-profilers/05-capstone-slow-service-to-evidence-fix.md

Unit 5 — Capstone drill: synthetic slow service → profiler → tracer → guarded optimisation

Compose **one** knowingly flawed service binary in `perf-lab/` exhibiting:

CPU bottleneck branch AND subtle blocking wait concurrently (mix signals)
potential memory retention OR goroutine buildup optional accent

You must traverse:

CPU `pprof` → hypothesis
`go tool trace` (optional) → validate scheduling / wait suspicion
Heap / goroutine profile if retention suspected
implement smallest effective change with before/after evidence
reject readability-destroying micro-optimisations without measured wins

Phase — 17-production-go-observability-and-containers

Directory: `17-production-go-observability-and-containers/`

Source: `17-production-go-observability-and-containers/01-scope-prod-service-24-by-7-ownership.md`

Unit 1 — Production mindset & `prod-service/` spine

Informative cadence: historically ~twelve deepening blocks aligning ~1–1.5 h/day authoring intent—ordering only.

Outcome metamorphosis

Stop celebrating “runs on my laptop.” Pursue artefacts proving **survivability 24/7**:

Structured logs with correlation identifiers
metrics (Prometheus exposition patterns ideologically—even if scraping infra stubbed mentally)
tracing (OpenTelemetry bridging multi-hop flows)
containers (Docker reproducibility—not hero builds)
graceful degradation signals (health vs readiness distinctions)
conceptual Kubernetes awareness (pods/deployments/services) without pretending admin mastery prematurely unless your role demands deepening elsewhere intentionally

Spine codebase: `prod-service/` (HTTP/gRPC hybrids acceptable if bounded—avoid scope explosion uncontrolled ethically warn learners responsibly).

Source: `17-production-go-observability-and-containers/02-docker-thinking-images-and-layers.md`

Unit 2 — Container basics: images, layers, reproducible builds

Outcomes

- Explain **layers** as cached build steps—and why careless `Dockerfile` ordering invalidates caches and slows CI.
- Distinguish **build toolchain image** vs **minimal runtime-only** carrier (expanded in Unit 3 with multi-stage patterns).

Practice

Produce a runnable image for `prod-service/` from a `golang` base (single-stage acceptable for learning), run the binary inside the container, and verify behaviour matches host builds.

Lab

Answer in prose:

- Why pinning **base image digests** matters for supply-chain reproducibility (even if you used a tag locally “for speed”).
 - What changes between “dev ergonomics Dockerfile” vs “delivery Dockerfile” mentality.
-

Source: 17-production-go-observability-and-containers/03-docker-multi-stage-minimal-runtime-image.md

Unit 3 — Multi-stage Docker builds: separate compile from runtime

Anti-pattern

Shipping a compiler, build caches, test tooling, and dev-only files in the same image you deploy.

Preferred shape (conceptual)

Stage A (builder): compile the Go binary reproducibly.

Stage B (runtime): copy the binary (+ any `embed` assets) into a small base image.

Practice

Refactor Unit 2's Dockerfile to multi-stage. Record **approximate image size change** (rough numbers are fine) and explain what disappeared from the final image layers.

Interview prompts

Alpine vs Debian vs distroless trade-offs—especially around DNS/TLS certs, libc quirks, debugging difficulty—stay honest and check current docs for your chosen base.

Source: 17-production-go-observability-and-containers/04-health-liveness-readiness-endpoints-and-semantics.md

Unit 4 — Liveness vs readiness endpoints (distinct Kubernetes signals)

Orchestrators distinguish:

Probe	Typical question
Liveness	Should this instance be restarted? (Use carefully—false positives restart-storm outages.)
Readiness	Should traffic be routed here right now? (Often reflects dependency health.)

Practice (`prod-service/`)

Implement:

```
GET /health (minimal "process alive" / cheap checks)
GET /ready (dependency checks-DB ping, migrations gate, critical cache dependency, etc.)
```

Simulate "database unavailable" (feature flag/env toggle) and show how `/ready` should fail **while** `/health` **can still succeed** if that matches your reliability policy.

Interview prompts

Why using **liveness** to detect downstream outages tends to produce restart storms instead of graceful traffic shedding.

Source: 17-production-go-observability-and-containers/05-graceful-shutdown-signals-and-draining.md

Unit 5 — Graceful shutdown: signals, draining, and worker collaboration

Production stops processes—rolling deploys, spot interruptions, human maintenance.

Pattern outline:

```
Listen for SIGINT/SIGTERM via signal.Notify
Stop accepting new HTTP connections (http.Server.Shutdown with context budget)
Finish in-flight requests / drain worker pools cooperatively
Flush logs / exporters if needed (bounded time)
exit cleanly
```

Practice

Extend a queue worker or HTTP server in `prod-service/` with shutdown that **does not** drop active work instantly.

Interview prompts

Draining vs hard kill trade-offs; how shutdown interacts with Kubernetes `terminationGracePeriodSeconds` (conceptual).

Source: 17-production-go-observability-and-containers/06-structured-logging-slog-and-correlation-fields.md

Unit 6 — Structured logging in production (`slog` baseline)

Replace ad-hoc prints with **keyed fields** and stable event names.

Minimum fields to internalise:

```
request_id
trace_id (or trace/span IDs once OTEL hooked)
service name / version
error classification fields (code, retryable bool) optional but valuable
```

Practice

Inject **request_id** at edge middleware; propagate into logs for a synthetic `Payment` API path.

Simulate a **500** and verify you can trace the request through logs without guessing.

Interview prompts

PII in logs; cardinality of dynamic keys; sampling vs full fidelity.

Source: `17-production-go-observability-and-containers/07-prometheus-metrics-red-and-service-sli-starter.md`

Unit 7 — Metrics with Prometheus conventions (RED basics)

Introduce counters/histograms summarising service behaviour—classical **RED** starter:

```
Rate (request rate)
Errors (error rate)
Duration (latency distribution)
```

Instrument `prod-service/` minimally:

```
http_requests_total{route,method,status}
http_request_duration_seconds_bucket (histogram)
optional business counters (payments_attempted_total, etc.)
```

Lab

Explain why **histogram buckets** must be chosen consciously (not infinite labels).

Interview prompts

High-cardinality label disasters; metric naming consistency; SLO thinking at high level.

Follow-on

Dashboard variables, alerting literacy, and false-positive hygiene for these same series are covered in **Unit 14**.

Source: `17-production-go-observability-and-containers/08-opentelemetry-tracing-spans-and-propagation.md`

Unit 8 — OpenTelemetry tracing across multi-hop flows

Trace a request across:

```
api → payment → inventory → queue publish (or gRPC hop)
```

Minimum concepts:

```
trace id propagation
span parent/child relationships
baggage cautions (don't smuggle secrets)
```

Practice

Add OTEL SDK wiring (HTTP server + outbound client spans). Verify a trace stitches IDs you already log (Unit 6).

Interview prompts

Sampling head vs tail; trace volume cost; debugging with traces vs logs vs metrics—when each wins.

Source: 17-production-go-observability-and-containers/09-kubernetes-backend-mindset-pods-deployments-services.md

Unit 9 — Kubernetes backend mindset (not admin exam)

You are not aiming to be the cluster SRE on day one—you need **enough vocabulary to reason about how your Go binary lives**.

Core nouns:

```
Pod
Deployment (rollout/replicas)
Service (cluster networking abstraction)
ConfigMap / Secret (config & sensitive material injection—still “secrets in etcd” caveats)
```

Practice (paper acceptable)

Sketch: `api` `pod(s)` → `Service` → dependency `database` (managed or not) showing where readiness gates traffic.

Interview prompts

Why Deployments create new ReplicaSets; what happens during rollouts to in-flight connections (high level).

Source: 17-production-go-observability-and-containers/10-configuration-and-secret-injection-ownership.md

Unit 10 — Configuration & secret ownership (no hard-coded production creds)

Anti-pattern: `DB_PASSWORD=admin` in source.

Preferred patterns (choose what matches your org, but understand trade-offs):

```
environment variables injected at runtime
mounted files / secret stores
12-factor style separation of config from code
```

Practice

Create `config/` package reading env with validation (fail fast on boot if misconfigured).

Interview prompts

Secret rotation; least privilege for reading secrets; avoiding logging config dumps.

Source: 17-production-go-observability-and-containers/11-incident-triage-metrics-traces-logs-triangle.md

Unit 11 — Incident triage: tie metrics, traces, and logs into one story

Synthetic incident:

CPU rising OR error rate rising OR latency rising (pick combo)

Your job is to narrate an investigation path:

Confirm user-visible symptom (dashboards / alerts)
Form hypotheses (dependency? GC? lock contention? retry storm? DB saturation?)
Validate with metrics first, then traces, then targeted logs
Apply fix with explicit trade-off note

Deliverable

One-page **post-incident style note** (even fictional) with timeline + evidence links you would have collected.

Source: `17-production-go-observability-and-containers/12-production-multi-surface-failure-lab.md`

Unit 12 — Production failure lab: break dependencies and prove observability pays off

Deliberately degrade:

database latency / errors
queue publish failures
worker stalls
aggressive client timeouts causing retry amplification

You must **detect** each class via your metrics/traces/logs story from Unit 11, not guess.

Rules

Each failure gets:

symptom → signal(s) used → root cause narrative → mitigation → prevention idea

No fix without evidence screenshots or metric panel descriptions—even textual “what I’d look at in Grafana/Jaeger” counts for learning artefacts now.

Source:

`17-production-go-observability-and-containers/13-capstone-payment-platform-production-shape.md`

Unit 13 — Capstone: `payment-platform/` as a production-shaped mini system

Compose (even partially stubbed honesty permitted if documented boundaries) `payment-platform/` with coherent services skeleton:

api
payment
inventory
worker (+ queue bridging earlier distributed/event areas lightly)

Must embed production primitives

Docker (multi-stage by default unless justified)

Metrics (Prometheus style)

Tracing (OpenTelemetry end-to-end at least api→one hop)

Structured logging correlation

/health + /ready split policy

Graceful shutdown

Synthetic chaos again (timeouts/failures) with evidence-driven fixes echoing Units 11-12

Interview checklist consolidation:

```
Docker multi-stage
graceful shutdown
metrics
tracing (spans propagation)
structured logging correlation ids
health vs readiness distinctions
OpenTelemetry vocabulary vs hand-wavy "we have traces" fluff debunk ethically
```

Source: 17-production-go-observability-and-containers/14-grafana-dashboards-variables-and-alerting-from-prometheus.md

Unit 14 — Grafana: turn Prometheus RED metrics into panels, variables, and honest alerts

Builds on **Unit 7 (Prometheus RED)**—same series should appear in Grafana as reusable operational views rather than anonymous scrape targets.

Learning outcomes

- Create a **small dashboard** with variables (`job`, `route`, or `instance`) so one board scales without copy-paste explosion.
- Choose **panel types** matching signal semantics (rates from counters, histograms as heatmaps/latency views where appropriate)—avoid lying “graphs of cumulative counters labelled as latency”.
- Encode **recording rules awareness** vs **heavy PromQL inside Grafana** trade-offs—when to simplify queries upstream.
- Author **two or three alert rules** (Prometheus alerting or Grafana-managed—pick one toolchain and justify) distinguishing **latency**, **error budget burn**, **absence of traffic** pitfalls.

Lab

Reuse metrics from `prod-service/` instrumentation from Unit 7 (or equivalently enumerated series). Produce:

```
one dashboard JSON export OR screenshot-linked narrative
alert definitions with explicit threshold rationale (not tribal “greater than 5%” cargo cult)
written note describing one false-positive you expect and how you'd tune observability—not silence permanently
```

Interview prompts

- **Cardinality dashboards:** how exploding label sets break cost and cognition.
- **Alert fatigue:** paging vs ticketing vs dashboards-only—explicit policy you’d propose for a hypothetical on-call rotation.

Acceptance criteria

One-page appendix describing **exactly how** you’d verify an alert firing was **correct** versus noise (which queries, what control chart or baseline idea—high level suffices).

Phase — 18-kubernetes-backend-advanced-placement-and-ingress

Directory: `18-kubernetes-backend-advanced-placement-and-ingress/`

Source: `18-kubernetes-backend-advanced-placement-and-ingress/01-hpa-and-autoscaling-metrics-basics.md`

Unit 1 — Horizontal Pod Autoscaling (HPA) — backend mental model

Backend engineers should know **why** a Deployment might scale replicas without treating it as witchcraft. Understand at a high level:

HPA reads metrics (often CPU/memory via metrics-server; custom metrics possible conceptually) adjusts desired replica count within min/max bounds subject to scheduling constraints

Practice (paper / local cluster optional)

Sketch an API service with variable RPS: decide which metric you would scale on (CPU alone is often a weak proxy for IO-bound services—articulate limitations).

Interview prompts

Why CPU autoscaling can amplify latency issues if the bottleneck is external (DB saturation).

Source: `18-kubernetes-backend-advanced-placement-and-ingress/02-resource-requests-limits-and-pod-eviction.md`

Unit 2 — Requests, limits, QoS, eviction, and noisy neighbour reality

Kubernetes uses **requests** (scheduling guarantees) and **limits** (caps) to approximate resource isolation. Study the mental model:

under-provisioned CPU request $\Rightarrow$ throttling risk
memory limit too tight $\Rightarrow$ OOMKill risk
no requests set $\Rightarrow$ scheduling lies + cluster instability

Understand **eviction** pressures when nodes run out of memory/disk—your pod may disappear even if “the code is fine.”

Interview prompts

“requests vs limits” in one sentence each; what **Guaranteed vs Burstable QoS** implies for eviction priority (conceptual—verify current kube docs when preparing).

Source: 18-kubernetes-backend-advanced-placement-and-ingress/03-rolling-deployments-and-rollbacks-backend-view.md

Unit 3 — Rolling deployments, restart scenarios, and rollout safety (backend angle)

Explore what happens during Deployment rollouts:

maxUnavailable / maxSurge interplay (high level)
ReplicaSet history and rollback idea
readiness preventing bad revisions from receiving traffic

Practice narrative

Describe failure mode: new version fails readiness—how does Kubernetes behave? How do you detect it before user impact if readiness is correct?

Interview prompts

Blue/green & canary contrasts (even if you only run rolling in practice).

Source: 18-kubernetes-backend-advanced-placement-and-ingress/04-ingress-edge-routing-overview.md

Unit 4 — Ingress: routing edge traffic to Services (conceptual sketch)

Ingress sits at the edge (cloud load balancers / ingress controllers vary—avoid single-vendor absolutism).

Understand:

DNS / TLS termination often lives near ingress layers
path-based routing to different Services
timeouts and body size limits commonly configured here (not only in app)

Practice (diagram)

Draw client → ingress → service → pods for your api, noting where health checks hit.

Interview prompts

Ingress controller vs API gateway overlap (high level); where WAF/rate limits often live (hint: Area 21 overlaps).

Source: 18-kubernetes-backend-advanced-placement-and-ingress/05-service-mesh-conceptual-tradeoffs.md

Unit 5 — Service mesh (conceptual trade-off, not a compulsory purchase)

A mesh adds a data plane proxy alongside workloads to standardise:

mTLS between services
traffic shifting
retries/timeouts policy (dangerous if blind)
observability hooks

Know the **tax**: operational complexity, latency overhead, debugging indirection.

Interview drill: when mesh benefits justify cost vs simpler explicit mTLS + ingress + good observability hygiene.

Source: `18-kubernetes-backend-advanced-placement-and-ingress/06-capstone-api-deployment-service-ingress-whiteboard.md`

Unit 6 — Capstone sketch: `api → Deployment → Service → Ingress`

Deliver a single cohesive diagram + short narrative describing:

```
how users reach your API
how rolling updates interact with readiness probes
where timeouts are configured (ingress vs service vs app client)
how you would surface golden signals (latency/errors/saturation) for the API's SLO story (even if panels are imaginary)
```

Treat this as interview whiteboard rehearsal, not a Helm chart competition.

Phase — 19-production-hardening-integration-and-rollout-discipline

Directory: 19-production-hardening-integration-and-rollout-discipline/

Source: 19-production-hardening-integration-and-rollout-discipline/01-probes-grace-periods-and-obs-contracts-pressure-test.md

Unit 1 — Probes, grace periods & observability “contracts” (pressure-test mindset)

Reuse ideas from Area 17, but now treat them as **operational contracts** that can misbehave.

Drill prompts (answer each in writing)

- When would `/health pass` while users see errors—and is that acceptable?
- How do misconfigured probes create **restart loops** vs **traffic black holes**?
- How does `terminationGracePeriodSeconds` relate to `Shutdown` timeouts in Go?
- What metrics/traces/logs would convince you **which** subsystem is saturated under load—not vibes?

Goal: articulate **signals you’d insist on before shipping** beyond “Dockerfile exists.”

Source: 19-production-hardening-integration-and-rollout-discipline/02-otel-metrics-tracing-correlation-policy.md

Unit 2 — OTEL + metrics + correlation policies (instrumentation without harming the service)

Treat observability rollout as engineering work with trade-offs:

- **Metrics:** naming consistency, cardinality discipline, sane histogram buckets.
- **Tracing:** propagate context across at least HTTP + any internal call you own; validate with a scripted multi-hop request.
- **Logs:** reuse the same correlation identifiers consistently (Areas `06/08` plus this hardening lens).

Lab

Instrument a tiny service upgrade path documenting **before vs after**:

- allocations / latency micro-benchmark snippet OR a short “pprof/quick bench” rationale if full bench impractical locally,
 - a checklist verifying sensitive fields are redacted (`Authorization`, tokens),
 - `/debug/pprof` never exposed unintentionally on public ingress (verify current security posture).
-

Source: 19-production-hardening-integration-and-rollout-discipline/03-config-logging-and-feature-flag-discipline.md

Unit 3 — Config ownership, structured logging refinement & safe feature flags

Goals

- Keep configuration **validated at startup** (`fail fast`).

- Separate **deployment config** vs **business feature toggles**.
- Understand feature flags pitfalls: unpredictable hot-path cost, auditing, accidental inconsistent evaluation across replicas.

Practice sketch

Extend `prod-service/` (or parallel mini service) reading:

```
required env vars
optional feature flag env/default policy explicitly documented per flag purpose
structured logs noting flag decisions only when useful (avoid spam)
```

Interview hooks: rollout strategies (percentage rollouts caveat high-level)—mesh/gateway splits optional mention without turning into vendor training.

Source: `19-production-hardening-integration-and-rollout-discipline/04-capstone-http-redis-postgres-grpc-obs-rollout.md`

Unit 4 — Capstone: hardened stack rehearsal (HTTP + Redis + Postgres + gRPC + metrics/trace + shutdown)

Assemble **one runnable slice** exercising previously separated concerns together—scope honesty permitted via stubs if each boundary labelled clearly (`// STUB:` with behaviour contract).

Mandatory integration checklist:

Capability	Requirement
HTTP API edge	timeouts + graceful shutdown hooked
Redis	caching / rate-limit backend / ephemeral lock illustrative—pick one purposeful use
Postgres	real query path OR documented stubbed interface bridging Area 08
gRPC collaborator	unary call respecting deadlines
Metrics	Prometheus scrape endpoint or pull exporter pattern
Tracing	OTEL exporting stub acceptable if propagation proven locally
Probes	<code>/health</code> + <code>/ready</code> policy written down
Docs	WHY certain toggles/timeouts chosen—not code-only submission

Produce a concise **risk note**: what fails first under load spike and what signals prove it.

Source: `19-production-hardening-integration-and-rollout-discipline/05-supply-chain-govulncheck-sbom-and-image-scanning-basics.md`

Unit 5 — Supply chain: `govulncheck`, SBOM discipline, and container image scrutiny

Companion to **fundamentals dependency honesty** (`go.mod` sums)—this turns “integrity on disk” into **continuous vulnerability posture**.

Goals

- Run `govulncheck` (or successor official vulnerability tooling) interpreting **called vs reachable** nuances—know why “CVE present in module graph” $\neq$ “exploitable in your binary”.

- Produce a **minimal SBOM** artefact during CI for one service (Syft/other standard generator acceptable) and state what you'd **actually** operationalise downstream (scanner feeds, approvals, SLA).
- Integrate **image scanning** basics (distroless vs debian slim trade-offs mirrored at policy level—not vendor tutorial) aligning base image pinning with rebuild cadence.

Concepts to internalise

- **Dependency audits** ↔ **running-artefact audits**: different scanners, different freshness contracts; reconcile both mentally before release gates.
- **Retracted modules** correlation with upgrade policy—tie back to semver/replace cautions earlier in the curriculum.

Practice sketch

Pick the **same** repo you hardened in Units 01-03. Extend CI conceptually (`bash` sketches acceptable):

```
module vulnerability scan producing non-zero informational output analysed
artifact SBOM archived with semver tag association narrative
Dockerfile rationale note: smallest change improving scanner noise vs debuggability
```

Interview prompts

- How do you prioritise patching when **severity is high but reachability uncertain** mid-sprint?
- Where does **checksum verification** (`go.sum` , image digests) end and **trust anchor** debates begin?

Acceptance criteria

Short **risk register** listing five rows: vuln tooling false negative class, SBOM staleness hazard, suppressed CVE governance risk, pinning versus drift, emergency rebuild communication path.

Phase — 20-interview-reasoning-and-end-to-end-labs

Directory: `20-interview-reasoning-and-end-to-end-labs/`

Source: `20-interview-reasoning-and-end-to-end-labs/01-concurrency-interview-drill-and-qa-cards.md`

Unit 1 — Concurrency interview lab: races, synchronisation, channels

Prompt A (classic)

```
counter := 0
for i := 0; i < 1000; i++ {
    go func() { counter++ }()
}
```

Explain data race, why results fluctuate, and fix using `sync.Mutex` or `atomic` with trade-off clarity (when atomics sufficient vs mutex clarity for invariants).

Prompt B (channels)

Given `jobs := make(chan int)` vs buffered variants—explain blocking semantics, suitable use-cases, dangers (forgotten readers, leaks).

Deliverable

Author **20 short Q&A cards** (even bullet form) covering mutex/channel/`select`/worker pool reasoning—your own words, not copy-paste trivia lists.

Source: `20-interview-reasoning-and-end-to-end-labs/02-interface-design-and-refactor-scenarios.md`

Unit 2 — Interface design interview lab: implicit satisfaction & segregation

Given:

```
type Logger interface { Log(msg string) }
```

Explain implicit implementation, why **small** interfaces ease testing and composition.

Extend with `io.Reader` style reasoning: large interfaces are debt.

Deliverable

Write **5 refactor scenarios** turning fat interfaces / concrete mega-structs into narrower seams without over-abstracting—justify each cut as if in staff review.

Source: `20-interview-reasoning-and-end-to-end-labs/03-context-cancellation-timeout-and-leak-fix.md`

Unit 3 — Context, cancellation & leak hunting lab

Start from `ctx := context.Background()` and layer:

```
WithTimeout
WithCancel
nested child contexts with shortened budgets
```

Explain propagation direction and common leak: goroutine ignoring `Done`.

Deliverable

Find & fix a deliberate worker leak in a toy repo you craft—document “signal used to confirm leak gone” (goroutine profile, runtime metrics story, etc.).

Source: `20-interview-reasoning-and-end-to-end-labs/04-memory-model-slices-maps-and-retention.md`

Unit 4 — Memory model interview: slices, maps, capacity, retention

Explain:

```
s := make([]int, 0, 100) // capacity vs length semantics
```

Discuss map growth, pointer semantics in map values (struct vs pointer values) at high level, and accidental slice backing array retention.

Deliverable

Three oral explanations (record notes) connecting **stack vs heap intuition** to evidence (escape logs / profiles) without myths.

Source:

`20-interview-reasoning-and-end-to-end-labs/05-race-detector-and-profiling-combined-incident.md`

Unit 5 — Race detector + profiling combined incident

Scenario:

```
CPU pegged + memory rising + workers “slow”
```

Use:

```
go test -race / go run -race on representative paths
pprof CPU + heap (and goroutine profile if suspect leaks)
```

Deliverable

Incident write-up: hypothesis list ordered by likelihood, evidence collected, fix, verification, trade-offs.

Source: `20-interview-reasoning-and-end-to-end-labs/06-distributed-scenario-payment-inventory-failure-chain.md`

Unit 6 — Distributed systems scenario lab: payment succeeds, inventory fails

Narrate and document:

retry policy (when safe vs dangerous)
idempotency & dedupe story
queue handoff / saga compensation framing (Areas 12/14)
eventual consistency symptom vs bug distinction

Deliverable: architectural note + sequence outline (textual fine) + explicit **open questions** you'd ask stakeholders.

Source: `20-interview-reasoning-and-end-to-end-labs/07-production-incident-triage-script.md`

Unit 7 — Production incident triage scripting (latency + errors + resource pressure)

Incident recipe:

Start from user-visible symptom
Query metrics hypotheses (SATURATION? ERRORS?)
Drill traces for slow spans
Align logs via correlation identifiers
Iterate until single dominant factor emerges OR honest multi-factor story accepted

Deliverable

Template you can reuse: fields for timeline, hypotheses, proofs, rollout.

Practice with synthetic DB slowdown + retry amplification scenario.

Source: `20-interview-reasoning-and-end-to-end-labs/08-architecture-whiteboard-payment-platform.md`

Unit 8 — Architecture whiteboard: payment platform services

Whiteboard (paper/digital) these services and their interactions:

API
payment
inventory
worker
notification

Annotate:

queue usage
retry/timeouts ownership
idempotency surfaces
observability hooks
data ownership boundaries (who owns which tables/events)

Voice-over ≤5 minutes without reading notes—record yourself if helpful.

Source:

`20-interview-reasoning-and-end-to-end-labs/09-enterprise-scale-stress-questions-first-drill.md`

Unit 9 — Enterprise-scale scenario drill (qualitative “100M orders” stress)

Given stylised scale signals:

```
worker slow, queue backlog growing, latency rising, memory rising
stack includes Go API + Redis + Postgres + queue
```

You must **ask questions first** (data shape, traffic pattern, hot keys, DB indexes, consumer lag, GC, retry storms) before proposing fixes.

Deliverable: structured Q&A + ranked hypotheses + measurement plan. No “rewrite the whole platform” without stating what evidence would justify it.

Source: `20-interview-reasoning-and-end-to-end-labs/10-final-staff-composite-payment-platform-lab.md`

Unit 10 — Final staff-style lab: `payment-platform/` under composite failures

Inject simultaneously (or staged if overwhelmed—document staging honestly):

```
duplicate payment attempts
timeouts & ambiguous responses
queue backlog growth
goroutine leak
CPU pegged profile
latency SLO breach symptoms
worker slowdown
```

You must produce **one consolidated incident report** including:

```
problem statement
symptoms timeline
investigation evidence (race? pprof? metrics? traces? logs?)
root cause(s)
fix + trade-offs
prevention / guardrails
```

Explicitly cross-link reasoning to earlier areas (concurrency, distributed, prod, perf) in the narrative—this is integration of *thinking*, not isolated trivia.

Phase — 21-backend-security-authn-authz-and-abuse

Directory: `21-backend-security-authn-authz-and-abuse/`

Source: `21-backend-security-authn-authz-and-abuse/01-threat-model-and-owasp-backend-mindset.md`

Unit 1 — Backend security threat model (OWASP-flavoured realism)

Security is not “add JWT and done.” Start from assets, trust boundaries, attackers, and blast radius.

Themes to internalise:

```
authn vs authz
secret lifecycle
rate limiting & abuse patterns
SSRF when fetching user-supplied URLs
mTLS for internal east-west (often mesh/ingress assisted)
```

Practice: write a 1-page mini threat model for `auth-service/` capstone path (Unit 10).

Source: `21-backend-security-authn-authz-and-abuse/02-jwt-vs-session-tradeoffs.md`

Unit 2 — JWT vs server-side sessions in API design

Compare trade-offs honestly:

Approach	Strengths	Pitfalls
JWT (often short-lived access)	stateless verification at edge	key rotation, revocation stories, fat tokens, algorithm confusion class of bugs historically
Sessions (server store)	simpler revocation	scalability, sticky sessions smell, store availability

Deliverable: verbal story for **logout**, **compromised token**, **key rotation** under each model.

Source: `21-backend-security-authn-authz-and-abuse/03-refresh-tokens-rotation-and-revocation.md`

Unit 3 — Refresh tokens: rotation, replay detection, storage trade-offs

Design a refresh flow that tolerates:

```
stolen refresh token scenario (rotation + reuse detection conceptually)
multiple devices
revocation requirements
```

Interview prompt: why refresh tokens deserve **tighter storage & transport** hygiene than access tokens.

Source: `21-backend-security-authn-authz-and-abuse/04-rbac-authorization-modelling.md`

Unit 4 — RBAC / authorisation modelling in services

Separate **authentication** from **authorisation**:

```
roles / permissions
resource-scoped checks ("can this user settle this payment?")
policy evaluation location (middleware vs service layer—both have trade-offs)
```

Practice: design a small permission matrix for admin vs merchant vs customer personas for `auth-service/`.

Source: `21-backend-security-authn-authz-and-abuse/05-api-security-basics-and-hardening-hooks.md`

Unit 5 — API security basics: headers, TLS posture, input mistrust

Cover essentials without turning into web pentest course duplicate:

```
TLS termination awareness (ingress/app)
HSTS / secure cookies mindset for browser clients (if applicable)
CORS is not authz
validate sizes & content types pragmatically
timing attack awareness for secret comparisons (crypto/subtle nuances high level)
```

Interview: difference between “encryption in transit” vs “authorisation correct”.

Source: `21-backend-security-authn-authz-and-abuse/06-secret-handling-config-and-least-privilege.md`

Unit 6 — Secret handling in Go services (no printf accidents)

Rules of thumb:

```
load secrets from env / mounted files / secret manager integration pattern for your org
never log secrets—even “temporarily”
rotate without redeploy thrash where possible (still often requires rolling restart practical honesty)
```

Practice: implement config struct with **redacted** `String()` for logs.

Source: `21-backend-security-authn-authz-and-abuse/07-rate-limiting-and-abuse-patterns.md`

Unit 7 — Rate limiting & abuse resistance (incl. idempotency abuse angles)

Design limits at edge (gateway) vs app (finer grained) trade spectrum.

Include **idempotency key abuse**: unbounded key cardinality, storage DoS, TTL strategy.

Deliverable: sketch policy for `/payments` endpoint bursts + duplicate retry storm friendliness.

Source: `21-backend-security-authn-authz-and-abuse/08-ssrf-and-untrusted-fetching.md`

Unit 8 — SSRF: fetching user-controlled URLs from the backend

Danger: background jobs hitting internal metadata endpoints because user supplied “URL to import.”

Mitigations sketch:

allow-lists / block private IP ranges / DNS rebinding awareness high level
separate network egress controls
never forward internal auth headers blindly

Interview: give a concrete Go `http.Client` footgun example and how you'd harden a fetcher.

Source: `21-backend-security-authn-authz-and-abuse/09-mutual-tls-internal-trust-model.md`

Unit 9 — mTLS conceptual (internal service-to-service trust)

Explain why east-west traffic sometimes uses **mutual TLS**:

service identity via certificates
encryption + authentication combined
often automated via PKI/mesh bridging Area18 mentions lightly

No lab mandate to provision full PKI—conceptual narration + handshake story sufficient unless you choose infra depth willingly.

Source: `21-backend-security-authn-authz-and-abuse/10-capstone-auth-service-security-shaped.md`

Unit 10 — Capstone sketch: minimal `auth-service/` with pragmatic security posture

Compose a deliberately small `auth-service/` exhibiting:

issue access token (whatever format chosen—JWT common)
refresh rotation skeleton (even if persisted in sqlite for practice)
RBAC middleware hook on a protected `/admin``-ish route stub
basic rate-limit counter (in-memory acceptable if documented non-prod)
SSRF-safe URL fetch stub OR explicit refusal pattern documented
secrets never logged demonstration test idea (table-driven asserting redaction fields)

Document known gaps honestly (no “production-ready” posture pretence without hardening checklist).

Phase — 22-testing-advanced-contracts-integration-and-mocks

Directory: 22-testing-advanced-contracts-integration-and-mocks/

Source: 22-testing-advanced-contracts-integration-and-mocks/01-table-driven-tests-and-subtest-discipline.md

Unit 1 — Table-driven tests as an explicit design tool

Use `tests := []struct{ ... }{ ... }` to model dimensions of behaviour:

- `name string` becomes the subtest label—optimise failure readability.
- Prefer **focused fields** (`giveInput`, `want`, `wantErr`) over giant anonymous structs obscuring failures.
- `t.Parallel()` only when isolation is guaranteed—avoid shared mutable fixtures.

Deliverable: convert one formerly copy-pasted test file into tables + parallel policy notes.

Source: 22-testing-advanced-contracts-integration-and-mocks/02-contract-testing-producer-consumer-safety.md

Unit 2 — Contract testing between services (producer/consumer safety net)

Consumers and producers deploy independently—schema drift hurts before integration tests notice if you only rely on “we run both in staging sometimes.”

Conceptual toolkit

Consumer-driven contracts / consumer expectations
Provider verification in CI
Versioned API contracts (OpenAPI / proto compatibility rules) as complementary guardrails

Deliverable

Pick one API boundary you own (even fake) and write:

- a contract scenario list (happy + breaking changes),
 - how you’d fail CI when the provider regresses,
 - what you still **cannot** catch (operational partial failures, performance)—state limits honestly.
-

Source: 22-testing-advanced-contracts-integration-and-mocks/03-integration-tests-and-database-semantics.md

Unit 3 — Integration testing strategy for repositories & databases

Move beyond “unit tests only” when SQL semantics matter:

transaction boundaries
constraint behaviour
migration compatibility smoke tests

Articulate cost: slower tests, flakiness risks—mitigate with containers or disciplined local fixtures.

Source: `22-testing-advanced-contracts-integration-and-mocks/04-testcontainers-realism-and-flake-hygiene.md`

Unit 4 — Testcontainers discipline (realism without endless flakes)

Practise patterns:

```
container lifecycle per package vs shared suite hooks trade-offs
migrations applied before tests deterministically
parallelism cautions hitting same ports—use ephemeral bindings
tear down hygiene even after panic fails (t.Cleanup idioms)
```

Deliverable: list your top five flake sources you'd defend against before blaming CI infra.

Source: `22-testing-advanced-contracts-integration-and-mocks/05-mock-ownership-and-test-doubles-discipline.md`

Unit 5 — Mock ownership: when interfaces help vs when mocks lie

Mocks should verify **interaction contracts sparingly**, not emulate entire subsystems that still melt in prod.

Interview stance:

- Prefer **fakes/stubs** for deterministic collaborators.
 - Use **mocks** for orchestration edges where you explicitly care about interaction shape.
 - Don't mock the universe—integration-test the seam where correctness is truly defined (`database/sql` , real migrations, HTTP to test server, clocks, RNG).
-

Source: `22-testing-advanced-contracts-integration-and-mocks/06-capstone-repository-integration-harness.md`

Unit 6 — Capstone: repository integration harness (confidence over line count)

Build a thin but real integration harness that proves `sqlx/Postgres-backed repositories` behave with real DB semantics—not only mocks.

Recommended checks (adapt to scope):

```
UNIQUE constraint violation maps to typed domain errors (no string contains hacks)
explicit transaction rollback on forced failure smoke path
migration apply on blank DB repeatable (or documented alternative fixture strategy)
performance sanity optional micro-benchmark not mandatory core here—confidence first
```

Write a half-page rationale: **why these tests earned CI time**.

Phase — 23-go-runtime-deep-dive-staff-level

Directory: `23-go-runtime-deep-dive-staff-level/`

Source: `23-go-runtime-deep-dive-staff-level/01-scheduler-gmp-preemption-and-syscall-blocking.md`

Unit 1 — Scheduler at staff depth: G/M/P (evidence over folklore)

Build on Areas `03`, `15`, `16` with sharper runtime vocabulary:

What staff-level clarity looks like

- Explain **why work stealing reduces idle P** under uneven goroutine churn—without pretending it removes all tail latency tails.
- State how **syscall blocking** can occupy an **M** and indirectly reduce CPU throughput for runnable goroutines (high-level mental model suffices; avoid OS-scheduler trivia unless you genuinely operate there).
- When discussing **preemption**, anchor claims to **current Go release/runtime documentation** rather than ageing blog summaries.

Interview drill:

- Differentiate scheduling pressure for **CPU-bound** vs **IO-bound** services.
 - Name at least three observability signals (CPU profile, trace, scheduler metrics if available) you'd use to validate a scheduling hypothesis.
-

Source: `23-go-runtime-deep-dive-staff-level/02-escape-analysis-and-allocation-tradeoffs-deep.md`

Unit 2 — Escape analysis: when the compiler sends work to the heap

Outcomes

- Use **escape diagnostics** (`go build -gcflags="-m"`) as a *hinting* tool, not a religion—profiles still win for hot paths.
- Predict common escape drivers: **interface boxing**, **variable captured by closure returned**, **slice/map elements referenced beyond stack lifetime**, **unsized values** passed through interfaces.
- Relate escapes to **GC pressure** and tail latency (Areas `15` / `16`).

Lab

Take one hot function from `perf-lab/` and:

1. capture `alloc_space` / `allocs/op` signal,
2. inspect escape report for that function,
3. attempt a *small* change that reduces allocations,
4. re-measure and document whether latency improved or readability regressed.

Interview prompt: “When would you ignore escape reports and trust `pprof` instead?”

Source: 23-go-runtime-deep-dive-staff-level/03-sync-pool-object-reuse-and-lifecycle-hazards.md

Unit 3 — `sync.Pool`: reuse without lying to yourself about safety

`sync.Pool` trades **allocation rate** against **risk** if pooled objects retain stale mutable state accidentally.

Rules of thumb

- Pool **allocation-heavy** ephemeral buffers / scratch structs—not long-lived authoritative domain records.
- Reset pooled objects meticulously before reuse; document invariants fiercely.
- Expect pooled objects **may disappear** anytime between GC cycles—never treat pooled storage as dependable cache correctness.

Practice: micro-benchmark pooling `[]byte` buffers for serialization hot path contrasting naive `make` churn—articulate concurrency hazards if pools cross goroutines wrongly.

Interview: summarise when pooling is ethically cancelled because correctness risk outweighs micro wins.

Source: 23-go-runtime-deep-dive-staff-level/04-memory-alignment-and-false-sharing.md

Unit 4 — Alignment, padding & false sharing (micro-architecture, measured)

Staff-level depth means you can name rare effects **and** stay honest about how often you really need them day to day.

Alignment & padding

Different types have different alignment requirements; the compiler inserts **padding** so fields land on valid addresses. Occasionally this matters for compacting hot structs—but don't turn this into cargo-cult reordering without evidence.

False sharing

Independent goroutines mutate values that still share a **cache line**, causing cache-line “ping-pong” even though the variables are logically unrelated.

Lab (benchmark-first)

1. Build two micro-benchmarks: **tight adjacent counters** vs **spacing/padding** variants.
2. Record results and noise caveats (thermal throttling, unrelated background load).
3. Write a short “would I ship this optimisation?” decision: only if tied to a measured production tail-latency/regression story—not because an interview question exists.

Interview prompt: how you'd distinguish false sharing from lock contention or allocator pressure using `pprof` / trace.

Source: 23-go-runtime-deep-dive-staff-level/05-capstone-pprof-microarchitecture-investigation.md

Unit 5 — Capstone: micro-architecture suspicion → evidence → guarded change

Compose a single performance mystery in `perf-lab/` with **plausible** confusion between:

lock contention vs false sharing vs allocation/GC noise vs scheduler wait (use trace)

You must:

form alternate hypotheses explicitly
select minimal measurement to falsify each (CPU profile, heap profile, trace, race if relevant)
pick the smallest behavioural change with before/after artefacts
document trade-offs (readability, API shape, pooling hazards)

Interview rehearsal: defend your decision path in ≤ 4 minutes without tooling open—prove you internalised the investigative recipe from Areas 15–16 and this staff unit.
